Data Architecture Reference

A measured reference for data architecture and state design

Edition 2 (draft)

Built from commit `study-04/harness-phase-status-integrated` (`0d23e7bbe2bd6b74c9027478f9295883debfce33`) — working tree true.
Typst 0.15.1 (image `sha256:032e292249bcd378480cc7c142cfa324b63ef8aadeb88d7e7230320c4c9c422f`).
Evidence registry `book/evidence/v6/claims.json`, digest `c6242b6004c531d5a045fb85e02c894f0984c3c76ec4c9f408d32b72e39cacdd`.

Edition and provenance

This is a generated artefact: Typst source plus the evidence registry, compiled in a pinned Podman image. **Every claim in this book must resolve to a claim in the active registry**, and every number must trace back to something actually measured in a report this book cites. Nothing here is hand-entered at build time; the fields below are injected from the build script.

Source commit	`0d23e7bbe2bd6b74c9027478f9295883debfce33`
Source describe	`study-04/harness-phase-status-integrated`
Working tree at build	true
Built at (UTC)	`2026-09-26T16:53:18Z`
Typst version	0.15.1
Typst image digest	`sha256:032e292249bcd378480cc7c142cfa324b63ef8aadeb88d7e7230320c4c9c422f`
Evidence registry	`book/evidence/v6/claims.json`
Evidence digest	`c6242b6004c531d5a045fb85e02c894f0984c3c76ec4c9f408d32b72e39cacdd`
Source tree hash	`fcd438e1e93544c386ff50af5c80776cb0a02245534bc20a1f2f36754ff7a4f0`

The book is assembled with Typst inside a pinned Podman image; the evidence index at the end is rendered from the registry file at build time.

Coverage gap

Environment and provenance limits travel with every number: all results come from one host (`host-zenbook-ux5406sa`) with shared cores and no real network between "cluster" nodes. Run tags mark the **producing code state**, not a tree containing the results. A reader reproducing a run must check out the run tag to get the code, then read the committed results directory.

1 How to read a number here

- Every **major-family** section follows the same progressive structure: quick choice, where it thrives and perishes, common scenarios, mechanism, costs, evidence and reproduction. Variant, scenario and concept chapters use the shape their role needs, because a case study and a reusable explanation are not the same thing.
- **Direct evidence** is measured in the cited run. **Mechanism evidence** demonstrates how something works without establishing a rate. **Analogy** is a labelled transfer to a family that was not measured. A **gap** callout is not one thing: each card names its own kind.
- The kinds of gap in use, read from the registry rather than restated here, because they close differently: **schema limitation** — the representation cannot answer the question, so no run would · **coverage gap** — the case simply has not been measured yet · **instrument gap** — the property could not be observed with the tools the study had · **unstable measurement** — the number moved between runs with no code change ·
- A claim can also be **partially superseded** — part of it has been replaced, the card names what moved to which successor, and the dimensions still open stay listed. A **retired** claim is no longer active and is not cited at all.
- Gaps are never null results: “not measured” must not be read as “would not change this”.
- No cross-study numeric ranking appears anywhere; no comparability record exists.

Contents

1 How to read a number here 3
2 How to choose 7
2.1 Step 0 — answerability before performance 9
2.2 Step 1 — place each fact once 9
2.3 Step 2 — name the contended surface 9
2.4 Step 3 — decide who maintains derived state 9
2.5 Step 4 — say how strong the evidence is 10
2.6 A worked decision 10
3 Major families 11
3.1 Normalized facts 11
3.2 Rolldown / duplicated data 12
3.3 Rollups / materialized aggregates 13
3.4 Embedded documents 14
3.5 Append-only history / snapshots 16
3.6 External read copies / caches 16
4 Minor variants 19
4.1 Measured controlled pairs 19
4.2 Named but not measured as controlled pairs 19
5 Cross-family comparisons 21
5.1 What a controlled pair looks like 21
5.2 Comparisons this book deliberately does not make 21
5.3 Direct evidence for the two family contrasts 21
5.4 What these six comparisons cannot decide 21
6 Topologies 22
6.1 What was measured 22
6.2 What topology does to a design 22
6.3 Open topology gaps 22
6.4 Boundaries 22
7 Scenarios 23
7.1 Ticket overbooking and refunds 23
7.2 Reserved seating and holds 23
7.3 Configuration portal 24
7.4 Donor portal with an external cache 24
7.5 Scenario transfers are labelled 25
7.6 Where a scenario stops 25
8 Evidence and reproduction 27
8.1 The provenance model 27
8.2 Reproducing the book itself 27
8.3 Reproducing a measured run 27
9 Part II — Concepts 28
10 Information placement 29
10.1 The ladder 29
10.2 Boundaries 30
10.3 The checklist 30
11 Indexes and hot rows 31
11.1 Indexes 31

11.2 Hot rows ... 31
11.3 Direct evidence ... 32
11.4 Boundaries ... 32
11.5 The checklist ... 32
12 Concurrency control ... 33
12.1 Optimistic (version-checked) ... 33
12.2 Pessimistic (lock-before-update) ... 33
12.3 The negative control matters here ... 34
12.4 Where the two designs cross ... 35
12.5 Boundaries ... 35
12.6 The checklist ... 35
13 Derived state and history ... 36
13.1 Maintenance is the design ... 36
13.2 History is answerability ... 37
13.3 Direct evidence ... 37
13.4 Boundaries ... 38
13.5 The checklist ... 38
14 Expiry and clock authority ... 39
14.1 What the corpus shows ... 39
14.2 What it cannot yet show ... 39
14.3 Boundaries ... 40
14.4 The checklist ... 40
15 Cache consistency ... 41
15.1 The flow: contract first, pattern last ... 41
15.2 The strict contract, written down ... 42
15.3 The named mechanisms, one primary failure mode each ... 43
15.4 What each mechanism is actually for ... 46
15.5 Freshness is a contract, not a speed knob ... 47
15.6 The gain, and what the gain leaves out ... 47
15.7 The failure mode, measured against the strict rule ... 47
15.8 Churn, engines and placement (2026-09-23) ... 49
15.9 What is still open ... 50
15.10 Boundaries ... 50
15.11 The checklist ... 50
16 Distributed placement ... 51
16.1 What the corpus measures ... 51
16.2 What it does not measure ... 51
16.3 Boundaries ... 52
16.4 The checklist ... 52
17 Reading benchmarks honestly ... 53
17.1 Six questions to ask of any number ... 53
17.2 Worked examples of a number that cannot be quoted ... 53
17.3 Before quoting a number ... 53
17.4 Boundaries ... 53
17.5 The checklist ... 54
18 Glossary ... 55
19 Evidence registry (active v6 claims) ... 58
19.1 normalized facts ... 58
19.2 rolldown / duplicated data ... 60
19.3 rollups / materialized aggregates ... 60

19.4 embedded documents 62
19.5 append-only history / snapshots 62
19.6 external read copies / caches 63
19.7 Retired predecessor claims 66

2 How to choose

CHAPTER · HOW TO CHOOSE

This chapter is the decision path. It front-loads the two questions that dominate every later trade-off: what must the schema be able to answer, and who arbitrates the contended row.

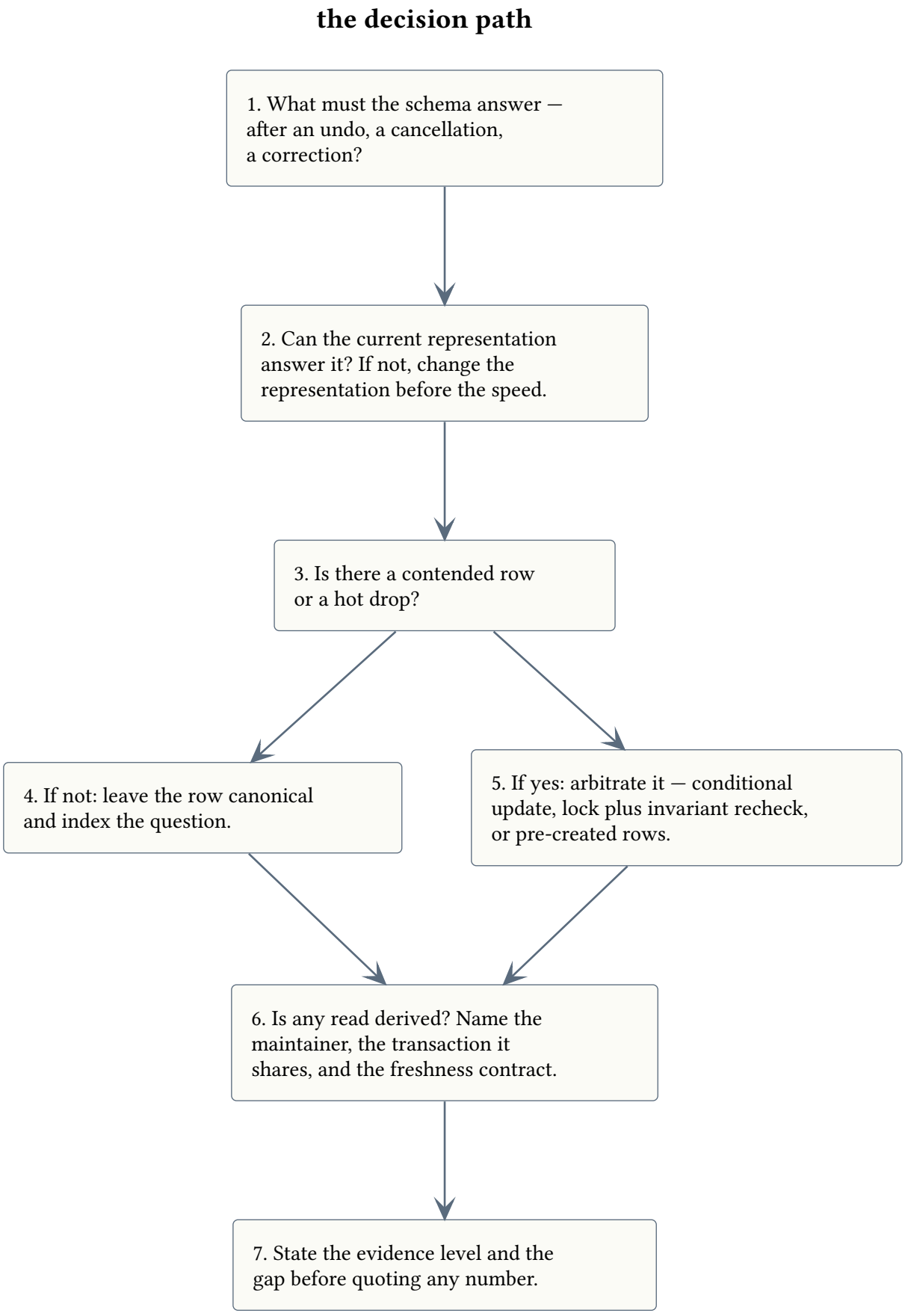


Figure 1: **The decision path, in the order the decisions bind.**

Form: decision tree · Evidence: conceptual illustration

Text equivalent. What must the schema answer — after an undo, a cancellation or a correction? If the current representation cannot answer it, change the representation before the speed. If a row is contended, arbitrate it: a conditional update, a lock plus an invariant recheck, or pre-created rows. If it is not, leave the row canonical and index the question. Name the maintainer of any derived read. State the evidence level and the gap before quoting a number. The figure orders the chapters' recommendations and asserts no measured value of its own.

2.1 Step 0 — answerability before performance

Direct evidence

Write down the questions the data must answer **after** an undo, a cancellation or a correction. Two measured answerability failures make this a rule rather than a preference: refunds were unanswerable in 14 of 15 ticketing designs, and no reserved-seating design could size a hold window because an expired hold is erased.

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33–35% more storage.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: answerability is a schema fact, not a timing; the +33–35% storage delta is a relation-size difference measured in the primary run and deterministic for the same dataset and design; it is not a repeated timed result; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap · schema limitation

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

If a required question is unanswerable, no index or cache fixes it. Choose a family that can answer it — usually append-only history — and accept its maintenance cost.

2.2 Step 1 — place each fact once

Walk the derive → index → materialize → copy/cache ladder (see the **information placement** concept) for each question. Prefer the lowest rung that meets the requirement, because each rung up adds maintenance, storage or a consistency contract.

2.3 Step 2 — name the contended surface

Find the rows many writers touch at once. If there is one, choose an arbitration strategy explicitly: pre-created units with compare-and-set, `SKIP LOCKED`, or a version-checked update. The counter-row design was 4–8x slower than pre-created seats and blocked unrelated edits.

2.4 Step 3 — decide who maintains derived state

For every rollup or copy, name the maintainer and the failure it must never have. A trigger makes the guarantee structural. Application maintenance kept writes near the reference in the measured configuration-portal workload, and its obligation is stated against where the derived state lives: when the base fact and the rollup are in the same database, update both in one transaction; when they cannot share one atomic transaction, name the delivery, idempotency and reconciliation mechanism that keeps the derived state recoverable. Only the second case is **publication**, because only there does a copy escape the transaction that wrote the original — leaving a moment when a reader can see the new value while the copy is still old. The measured trigger cost was 4.0–5.7x on whole-configuration replacement across three runs (median 4.94x) — a range rather than a single figure, because repeating the measurement inside that configuration gave results that disagreed with each other by more than we would like.

2.5 Step 4 — say how strong the evidence is

Every recommendation states how well it is supported, what it could not separate, and what has not been measured. A recommendation that does not say what is missing claims more than the evidence allows.

2.6 A worked decision

- A ticketing system must never oversell and must report refunds → append-only ledger plus a pre-created-seat arbiter, +33–35% storage.
- A configuration portal reads its overview far more than it writes → application-maintained rollup; a trigger only if a forgotten write is unacceptable.
- A public donor portal serves warm profiles → cache with a named contract and a publication fence against the stale-fill race (a reader republishing S0 after a writer commits S1 and invalidates), accepting that the cache's budget is outside the `db-only` comparison.

3 Major families

CHAPTER · MAJOR FAMILIES

A family is a way of deciding where a fact lives. The six here are fixed by the goal; each major-family section follows the same progressive structure, and the mechanism chapters carry the reusable explanations. Every cost below is a measured claim, labelled **direct** when it comes from the cited run or **analogy** when it is a transfer.

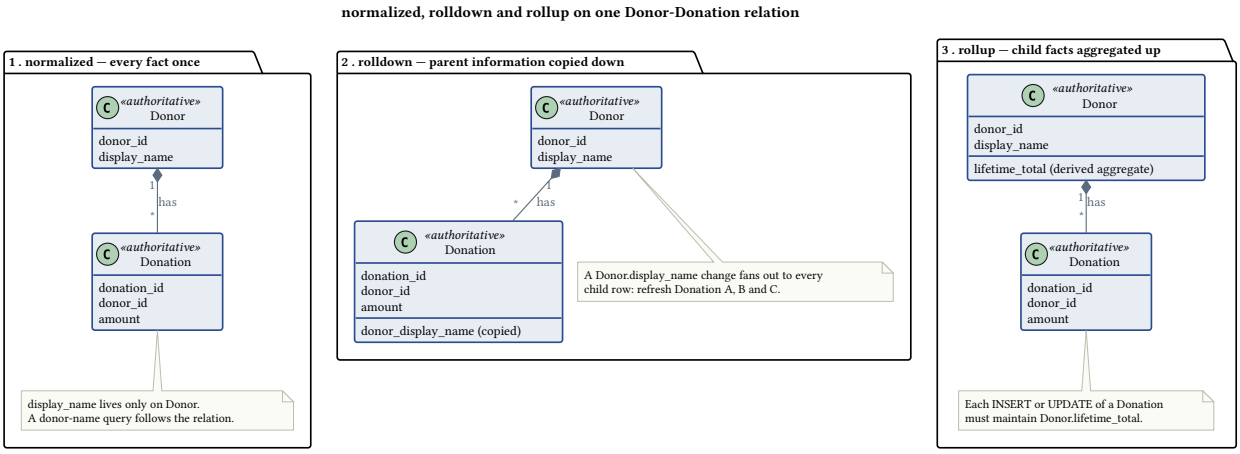


Figure 2: **Normalized, rolldown and rollup on one parent-and-child relation.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Three panels over one donor-to-many-donations relation. **Normalized** keeps every fact once: `display_name` lives only on `Donor`, so a query that needs a donation with its donor name follows the relation. **Rolldown** copies the parent value down onto the child (`donation.donor_display_name`), so the join disappears and a parent change fans out to every child row. **Rollup** aggregates child facts up onto the parent (`donor.lifetime_total`), so parent reads are local and every contributing child change must maintain the aggregate. The figure maps families 1–3 and carries no measured value.

3.1 Normalized facts

3.1.1 Quick choice in plain language

Keep each fact in one place and let queries assemble the answer. Choose it unless a measured question says otherwise.

3.1.2 Where it thrives and where it perishes

Thrives when questions are answerable from the rows themselves and writes are spread across many parents. Perishes on a hot row, or when the required question was never modelled.

3.1.3 Common scenarios

Order lines, ticket sales, configuration entries — the canonical transactional facts. Normalization decides **where those facts live**; it does not by itself prevent overbooking or double-selling, which needs a declared constraint plus a concurrency strategy that acts on it.

3.1.4 Mechanism

See the **indexes and hot rows** concept and the **concurrency control** concept. Indexes answer expressible questions; arbitration answers contended rows.

3.1.5 Costs

Reads pay per query; writes are cheap **in the measured cells of this corpus**, not as a law; storage is the baseline; correctness depends on the constraint you actually declared; operations are ordinary.

3.1.6 Direct evidence

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-01-study01-d2d3-copied-key-package

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4–8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: studies/02-ticket-booking/reports/20260913T021206Z.md · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

3.1.7 Boundaries and reproduction

Every ratio on this page comes from one run of the experiment, on one machine, so read each as a direction rather than a stable number. Reproduce it from the run tags in the cards, and check the note on the card for any part of that run that failed or was invalid before quoting a percentage.

3.2 Rolldown / duplicated data

3.2.1 Quick choice in plain language

Copy a parent value onto the child to remove a join. Do it when the copied value is small, stable enough, and the join is on the hot path.

3.2.2 Where it thrives and where it perishes

Thrives on read-heavy workloads with a bounded parent set. Perishes when the copied value changes often, because every change becomes a fan-out write.

3.2.3 Common scenarios

A donation row carrying its donor’s display name; an order line carrying its product’s category.

3.2.4 Mechanism

See the **information placement** concept. The measured pair is a package — the copied key plus its SQL and indexes — so attribute the gain to the package, not to the column alone.

3.2.5 Costs

Reads improve; writes gain an index and a copy to maintain; storage grows; correctness needs a rule for the stale copy; operations must backfill after a schema change.

3.2.6 Direct evidence

v2-05-d3-read-advantage-repeated

rolldown / duplicated data · repeated_controlled

D3's (d3_flattened_fk) read advantage over the normalized indexed design survives five fresh loads; the recency SQL and actual heap avoidance explain the gain, while its insert cost remains imprecise.

Trials: 1 whole-run replication · 5 fresh-load trials per cell · 1 inner iteration

Limits: five fresh-load trials per cell, one session; the insert-cost estimate is imprecise and must not be quoted as a ratio

Source: studies/01-charity-tree/reports/20260913T125342Z-v3.md · run 20260913T125342Z-v3 · tag run/01-charity-tree/20260913T125342Z-v3

3.2.7 Boundaries and reproduction

One family contrast is not a general law. The copied-key comparison changes more than the copied key, so it cannot be attributed to that decision on its own (`conf-01`), and where the cluster physically put the data was never verified (`v4-gap-01`).

3.3 Rollups / materialized aggregates

3.3.1 Quick choice in plain language

Store the total on the parent. Use it when an aggregate is read far more often than its children change, and be explicit about who maintains it.

3.3.2 Where it thrives and where it perishes

Thrives on dashboards and parent-level reads. Perishes when child writes dominate, or when a write path can forget the rollup.

3.3.3 Common scenarios

Donor lifetime totals, event ticket counts, a configuration portal's overview.

3.3.4 Mechanism

See the **derived state and history** concept. Trigger-maintained is a structural guarantee at a write cost. An application-maintained rollup is transactional first: where the base fact and the rollup share a database, update both in one transaction. Where they cannot share the authoritative transaction, the copy crosses that boundary and becomes **publication** — name the delivery, idempotency and reconciliation that keep it recoverable. In the measured configuration-portal workload application maintenance kept writes near the reference; that is a scope, not a law.

3.3.5 Costs

Reads win big; writes pay maintenance; storage grows modestly; correctness is the whole game; operations must rebuild the rollup after a bulk load.

3.3.6 Direct evidence

v2-02-rollup-family

rollups / materialized aggregates · single_run_directional

Parent rollups win the aggregate read score (D4 rollup/trg 73.8x and D5 rollup/app 69.9x against D1 minimal) and pay for it on inserts (2.6k/s and 2.1k/s against 6.2k/s).

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run, one trial per cell; PostgreSQL 1 node numbers shown; YugabyteDB in the same report; the write cost is per-insert on a fresh load

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-13-trigger-rollup-cost

rollups / materialized aggregates · run_replicated

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 4.0–5.7x the application-maintained rollup (n4_rollup_app) on whole-configuration replacement across three randomized-order runs (median 4.94x: n4 518–635 ops/s against n3 91–159 ops/s), and the trigger's latency tail is 22–46x worse (p99 61–109 ms against 2.2–2.7 ms); the trigger cell's own trials disagree by 35–115%, so this is a range over an unstable cell, and the earlier single-run 6.85x figure did not reproduce.

Trials: 3 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale; three whole runs, three trials per cell in each; run-level replication only: separate runs on one machine, not hardware-independent, and still exposed to this host's measurement hazards; the trigger cell's own trials disagreed by 35–115% across the three runs, above the ~20% precision rule, so only the direction and an order-of-magnitude range are supported; the ratio is not a point estimate; the ~15x read gain belongs to the earlier single run and was not re-measured in the replication; the earlier single-run 6.85x figure did not reproduce and must not be quoted; the trigger's product is a structural guarantee, not speed; the three runs realized only two distinct design orders (seeds 202 and 303 coincided, and in the sibling read tranche all three seeds coincided); randomized order describes the method, and the realized order is recorded in each manifest; the penalty is size-dependent, not a constant: crossing the same pair with entries per installed product gives 2.07x at 10 entries, 4.38x at 60 and 24.20x at 240, so this ratio holds at the small scale default of 60 entries and must not be quoted as a property of the design without the size (v2–18-maintenance-size-interaction)

Source: studies/04-configuration-portal/reports/20260925T190831Z.md · run 20260925T190831Z · tag run/04-configuration-portal/20260925T190831Z

v2-04-recency-maintained-index

rollups / materialized aggregates · single_run_directional

The recency flag works, but an index on the rollup column answers the same question faster: comparing d20_recency_flag with d22_recency_rollup_idx in the same run gives the reported read speedup on PostgreSQL and YugabyteDB.

Trials: 1 whole-run replication · 3 fresh-load trials per cell · 1 inner iteration

Confounds: conf-02-study01-d20d22-write-axis

Limits: the write comparison is confounded: d22 also maintains D4's multi-column rollup package, so its write cost is not the index alone; three trials per cell; claim-02's v1 trials field was recorded as 1 and is corrected here

Source: studies/01-charity-tree/reports/20260916T090036Z-v3.md · run 20260916T090036Z-v3 · tag run/01-charity-tree/20260916T090036Z-v3

3.3.7 Boundaries and reproduction

We can trust the read comparison, but not the write comparison. The design with the recency index also performs the rollup maintenance, so the experiment cannot tell us how much of the write cost comes from the index itself. In experimental terms, this write comparison is confounded (`conf-02`).

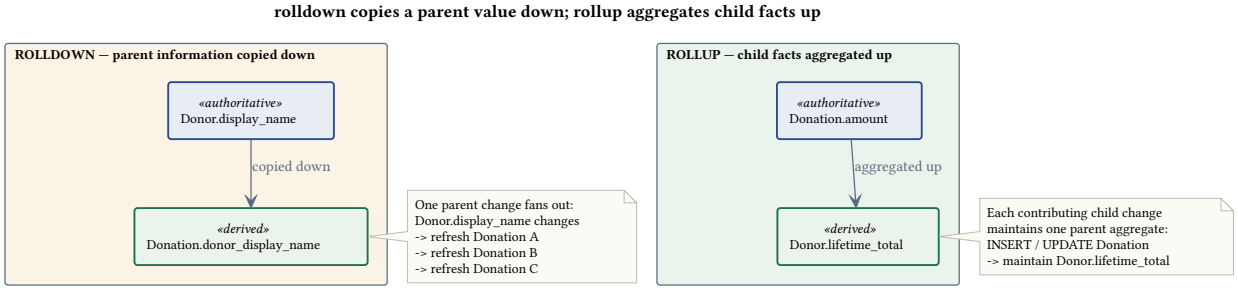


Figure 3: **Rolldown fans a parent change out; rollup aggregates a child change up.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** The two directions are opposites. **Rolldown** copies parent information down, so a Donor value that changes must be refreshed on Donation A, B and C — a fan-out write. **Rollup** aggregates child facts up, so an insert or update on `Donation` maintains one `Donor.lifetime_total` — a single parent write per contributing child change. The read gain and the maintenance cost point in opposite directions, and the figure asserts no rate.

3.4 Embedded documents

3.4.1 Quick choice in plain language

Put the children inside the parent when they are read together and bounded in size. It is a read-locality choice, not a database-family choice.

3.4.2 Where it thrives and where it perishes

Thrives when a bounded child set is always read with its parent. Perishes when children are queried independently or grow without bound.

3.4.3 Common scenarios

A reservation's seat map, a configuration file rendered whole.

3.4.4 Mechanism

See the **information placement** concept. A section document answered a 100k-seat map 4.5–11x faster than per-seat rows on YugabyteDB; a sharded variant collapsed point lookups to 26 ops/s.

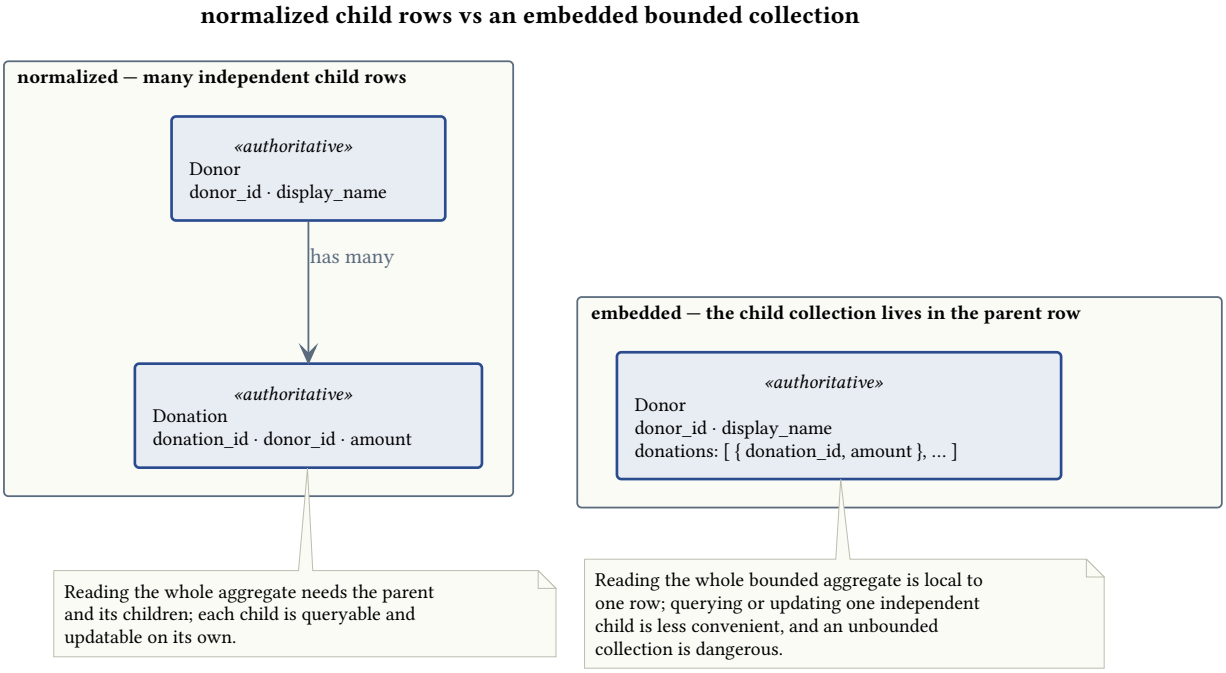


Figure 4: **Normalized child rows beside a bounded child collection inside the parent.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Left, normalized child rows: the parent and its children are independent rows, so the whole aggregate and each single child are both addressable, at the cost of a join or an aggregate read. Right, the same bounded collection stored inside the parent row: reading the whole aggregate is local to one row, while querying or updating one independent child is less convenient and an unbounded collection is dangerous. The figure contrasts the placements and asserts no rate.

3.4.5 Costs

Reads of the whole child set improve; point access to one child can get much worse; storage is similar; correctness needs a bounded-growth rule; operations can be awkward for partial updates.

3.4.6 Direct evidence

v2-03-embedding-not-ahead

embedded documents · single_run_directional

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: 2.4x D1 minimal's read score, below both indexed designs, with a 0.57x insert rate.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5–11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-03-study03-layout-family-package

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

3.4.7 Boundaries and reproduction

Embedding here means a JSON/array child column in PostgreSQL or a document row in YugabyteDB — a labelled **analogy** to a native document database, which was not measured here (`v2-gap-06`). The comparison of a table of rows against a stored document changes both designs at once, so it contrasts two whole ways of organising the data rather than one decision (`conf-03`).

3.5 Append-only history / snapshots

3.5.1 Quick choice in plain language

Never overwrite the fact; append what happened. Choose it when a question must survive an undo.

3.5.2 Where it thrives and where it perishes

Thrives wherever refunds, cancellations or corrections are in scope. Perishes under storage limits with no retention policy.

3.5.3 Common scenarios

Refund reporting, audit trails, last-purchase questions after a cancellation.

3.5.4 Mechanism

See the **derived state and history** concept and the **expiry and clock authority** concept.

3.5.5 Costs

Reads need a report over history; writes append and storage grows (about 33–35% more in the ticketing ledger); **answerability improves** — this family is the answerability fix, because retained history keeps enough information to answer across refunds, cancellations and corrections. An append-only history does not itself fence concurrent writers, order events, deduplicate, reject invalid events or provide atomicity; concurrency and arbitration must be specified separately. Operations need retention.

3.5.6 Direct evidence

v2-07-refund-answerability-and-storage · single_run_directional · answerability is a schema fact, not a timing; the +33–35% storage delta is a relation-size difference measured in the primary run and deterministic for the same dataset and design; it is not a repeated timed result; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded · full card in the evidence registry index

v2-08-ledger-race-cost-128-buyers

append-only history / snapshots · single_run_directional

At 128 buyers the ledger costs 1.1–2.3x of the sell-out race on PostgreSQL and 1.8–2.2x on YugabyteDB with tight spreads, 2.1–2.5x on the refund itself, and not one under-booked seat anywhere; it delays the organiser's own edit by up to 5x.

Trials: 1 whole-run replication · 3 fresh-load trials per cell · 128 inner iterations

Confounds: conf-05-study02-buyer-arms-disagree

Limits: one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier

Source: studies/02-ticket-booking/reports/20260921T205212Z.md · run 20260921T205212Z · tag run/02-ticket-booking/20260921T205212Z

3.5.7 Boundaries and reproduction

The PostgreSQL ledger cost is order-sensitive and unsettled (`v2-09`); the load multiplier is not published (`v2-gap-01`). Quote the 128-buyer scoped result, never a pooled ratio.

3.6 External read copies / caches

3.6.1 Quick choice in plain language

Serve the answer from beside the database, with a stated freshness contract. Do it for warm, cacheable, read-heavy questions.

3.6.2 Where it thrives and where it perishes

Thrives on repeated reads of stable keys. Perishes when many instances share a cache under relaxed freshness without a publication fence. The fence closes one specific race: a reader that snapshotted committed state S0 republishes it after a writer commits S1 and invalidates.

3.6.3 Common scenarios

A donor portal's hot profile reads, a public dashboard over changing totals.

3.6.4 Mechanism

See the **cache consistency** concept for the guarantee-first flow: name the contract, establish change-observation completeness, read the source's capabilities, then choose topology (process-local versus shared) and the fill path (cache-aside versus write-through). Topology and fill path are not guarantees; performance controls (fill lease, TTL, early expiry) are added separately.

3.6.5 Costs

Reads gain 2.2–3.5x and p99 drops an order of magnitude; a strict protocol adds a write-path fence; storage moves to the cache (whose budget is outside the `db-only` comparison); operations gain a new failure domain.

3.6.6 Direct evidence

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2–3.5x in throughput and an order of magnitude in p99 (15.8–27.1 ms to 0.8–2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-04-study05-db-only-cache

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); this 2.2–3.5x range is supported only by the PostgreSQL single-node + Redis cells; do not transfer the range to YugabyteDB, to a cluster, or to open-loop demand

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-04-study05-db-only-cache

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. This claim measures read throughput only: it does not quantify the write-path cost of a strict protocol, and it must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-06-claim12-full-noise-range, conf-04-study05-db-only-cache

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v3-03-equal-total-framing-labelled

external read copies / caches · single_run_directional

Cache resource accounting now has a separately labelled equal-total arm: with the same design and topology the warm cacheable read is 11k ops/s under equal-total framing and 8.0k under db-only, the framing is recorded in every result, and the two arms are never pooled.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run per framing, one host, scale small; the two arms are single trials; the value of the arm is the label and separation, not a throughput ratio; the difference is inside the run's spread and is not attributed

Source: studies/05-cache-consistency/reports/20260923T1150Z-res2-equaltotal-pg.md · run 20260923T1150Z-res2-equaltotal-pg · tag run/05-cache-consistency/20260923T1150Z-res2-equaltotal-pg

3.6.7 Boundaries and reproduction

Redis was measured as a cache, never as an authoritative store. The gain is measured against a database-only reference point, so it cannot be attributed to the cache by itself (`conf-04`), and the comparison against equal total resources per side is labelled separately. The failure with three instances shows the mechanism at work rather than a rate to predict from, and the in-process designs' zero wrong reads are a bypass result (`v2-15`), not safety.

4 Minor variants

CHAPTER · MINOR VARIANTS

A minor variant is one controlled change inside a family. The corpus has measured pairs for some; the rest are named with their evidence status so a reader can tell the difference.

4.1 Measured controlled pairs

4.1.1 Index selection

Read score — used from here on — is this corpus's blended within-run composite of a family's read queries. It compares variants inside one run and nothing else: it is not the throughput or the latency of any single query, and it must not be quoted across runs.

Adding the query index was the first large read win in the charity tree (D2 indexed, 13.3x the minimal design's read score); flattening the foreign key reached 20.5x.

v2-01-normalized-index-first · single_run_directional · PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query · full card in the evidence registry index

4.1.2 Optimistic versus pessimistic concurrency

Under one hot key and 16 writers, lock-before-update delivered 1.76x the version-checked design — an upper bound, because the run could not separate the lock's advantage from extra per-retry work. An unchecked read-modify-write lost 93.7% of acknowledged updates.

v2-12-lock-vs-version-upper-bound · single_run_directional · one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve · full card in the evidence registry index

4.1.3 Trigger versus application maintenance

Trigger-maintained rollups cost 4.0–5.7x on whole-configuration replacement across three randomized-order runs (median 4.94x), and their latency tail is 22–46x worse (p99 61–109 ms against 2.2–2.7 ms); the application rollup kept writes near the reference. Repeating the measurement inside that one configuration gave 35–115% disagreement, so quote the range and never a point.

v2-13-trigger-rollup-cost · run_replicated · PostgreSQL single node, small scale; three whole runs, three trials per cell in each; run-level replication only: separate runs on one machine, not hardware-independent, and still exposed to this host's measurement hazards; the trigger cell's own trials disagreed by 35–115% across the three runs, above the ~20% precision rule, so only the direction and an order-of-magnitude range are supported; the ratio is not a point estimate; the ~15x read gain belongs to the earlier single run and was not re-measured in the replication; the earlier single-run 6.85x figure did not reproduce and must not be quoted; the trigger's product is a structural guarantee, not speed; the three runs realized only two distinct design orders (seeds 202 and 303 coincided, and in the sibling read tranche all three seeds coincided); randomized order describes the method, and the realized order is recorded in each manifest; the penalty is size-dependent, not a constant: crossing the same pair with entries per installed product gives 2.07x at 10 entries, 4.38x at 60 and 24.20x at 240, so this ratio holds at the small scale default of 60 entries and must not be quoted as a property of the design without the size (v2–18-maintenance-size-interaction) · full card in the evidence registry index

4.1.4 Strict versus relaxed freshness

Strict after acknowledgement means a read that starts after a write to the same key is acknowledged never returns an older committed value. **Relaxed** is a family of weaker contracts, not one contract: compared with the strict promise just stated, a relaxed contract may permit a committed but older value according to its declared semantics. Only a **bounded-staleness** member declares a maximum Δ; eventual and best-effort freshness need not, and the relaxed contracts measured here declared no maximum, so they are unbounded-relaxed. No member permits a dirty, torn or impossible value. Every controlled pair differed by −14.2% to +12.3% in read throughput — noise. The visible cost of strictness is on the write path, where a publication fence is added against the stale-fill race (a reader republishing S0 after a writer commits S1 and invalidates).

v2-16-strict-freshness-read-cost · single_run_directional · read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop · full card in the evidence registry index

4.2 Named but not measured as controlled pairs

- **Full versus bounded embedding.** Embedding was measured as a family, not against a bounded variant.

- **Locks / CAS / isolation levels.** The corpus has CAS and lock designs, but not an isolation-level sweep under one invariant.
- **Pre-created versus on-demand inventory.** Measured in ticketing as a family contrast with arbitration, not isolated from concurrency strategy.
- **Cache-aside versus write-through.** Both appear; the three-instance failure is about fencing, not about the two patterns head-to-head.

Coverage gap

No variant above is a cross-study ranking, and none is an equal-total-resource comparison. Treat the unmeasured variants as open, not as null results.

5 Cross-family comparisons

CHAPTER · CROSS-FAMILY COMPARISONS

Comparisons across families are where over-claiming happens. The rule here: a pair is quoted only when one decision changed, and every comparison carries its confounds on the page.

5.1 What a controlled pair looks like

- **One decision changed.** The charity tree's index and rolldown pairs each change one placement decision; their confounds are named (`conf-01`).
- **Same workload and resources.** Logical data, operations and per-node budget held constant.
- **Controls present and fired.** A pair whose negative control did not fire is weaker than it looks.

5.2 Comparisons this book deliberately does not make

- **No cross-study numeric ranking.** No comparability record exists, so ratios from different runs are not pooled.
- **No equal-total-resource ranking.** Single-versus-cluster answers "add machines", not "same budget".
- **No native-family transfer.** JSONB/state is not a native document store and Redis was measured only as a cache; such a transfer is analogy, labelled as such.

5.3 Direct evidence for the two family contrasts

v2-03-embedding-not-ahead · single_run_directional · one run, one trial; embedding here means a JSON/array child column, not a native document database · full card in the evidence registry index

v2-11-section-seat-map · single_run_directional · one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable · full card in the evidence registry index

5.4 What these six comparisons cannot decide

Each item below is a comparison the book shows, followed by the reason it cannot settle the question it looks like it settles. The reason comes first because it is the useful part; the short label in brackets is only how the rest of the book refers back to it.

- **A copied key carries more than the key.** The D2⟶D3 rolldown compares two whole designs, so the read gain it measures includes the copied column, the index that supports it and the changed SQL — not the copied column on its own. [**`conf-01`**]
- **The recency pair's write cost is not only about recency.** The design with the recency index also does all of the rollup maintenance, so the experiment cannot say how much of the write cost comes from the index itself. Quote its read comparison, not its write cost. [**`conf-02`**]
- **A table of rows against a stored document is a whole-family comparison.** The seat map differs both in storage shape and in how the data is organised, so any difference belongs to the two designs together, not to one decision inside them. [**`conf-03`**]
- **The cache gain was measured against one baseline.** The reference point was the database alone, so the measured gain cannot be attributed to the cache by itself. [**`conf-04`**]
- **The two buyer models swap places.** On PostgreSQL the two designs change order depending on the workload, so neither can be reported as the faster one. [**`conf-05`**]
- **The strict-freshness numbers run in both directions.** The range goes from −14.2% to +12.3%, so quoting either end alone would misrepresent the result; the whole range travels with any summary. [**`conf-06`**]

6 Topologies

CHAPTER · TOPOLOGIES

Topology is a separate axis from design family: how many nodes there are, how many copies of the data exist, where those copies sit, how clients are routed to them, how resources are divided between the two sides of a comparison, what the network does, and what happens when a node fails. A design that is best on one node is not thereby best on three.

6.1 What was measured

Studies 01–03 run PostgreSQL single-node and YugabyteDB single-node (RF=1) and 3-node (RF=3). Each measured configuration is capped at a documented per-node budget, and every run manifest records it. The comparison is therefore "the same budget on every node", not "the same total resources on both sides".

6.2 What topology does to a design

In the measured YugabyteDB RF=3 topology, replication puts consensus work on the foreground write path, so write-heavy designs move differently from read-heavy ones there; asynchronous replication would not necessarily impose that. The section-document experiment shows that one **layout package** can affect two access patterns in opposite directions: the sharded variant answered the 100k-seat whole-map read 4.5–11x faster than a row design while collapsing point lookups to 26 ops/s. Because that package changes representation and placement together, it isolates no pure topology effect; the lesson is that a layout has to be measured again under each topology and access pattern you care about.

v2-11-section-seat-map · single_run_directional · one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable · full card in the evidence registry index

6.3 Open topology gaps

Coverage gap

Physical placement is unverified (`v4-gap-01`) and endpoint distribution is measured only for the 2026-09-23 Study 05 three-node cells (`v4-gap-02`); the earlier multi-node cells reach the cluster through a single endpoint, and there is no real network between "cluster" nodes. Multi-node results here are placement experiments, not distributed-systems results.

6.4 Boundaries

Do not scale these numbers to production capacity. The nodes share one host's eight cores, are CPU-throttled, and communicate over a loopback-class path.

7 Scenarios

CHAPTER · SCENARIOS

A scenario is a set of questions plus a workload shape. The same family can be right in one and wrong in another, so each scenario below states its answerability question and its evidence status.

7.1 Ticket overbooking and refunds

Never oversell, then report refunds. Pre-created seats with compare-and-set or `SKIP LOCKED` sold 4–8x faster than a counter row; a ledger made refunds answerable at +33–35% storage. The ratio between the two ways of arbitrating a seat at 128 simultaneous buyers is 1.1–2.3x on PostgreSQL, and the measurements at small scale disagree with each other, so that size does not settle which is faster.

v2-06-hot-drop-designs · single_run_directional · one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only · full card in the evidence registry index

v2-08-ledger-race-cost-128-buyers · single_run_directional · one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier · full card in the evidence registry index

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92–283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7–5.7x (not the headline's narrower 1.7–2.1x).

Trials: 1 whole-run replication · 3 fresh-load trials per cell · 32 inner iterations

Confounds: conf-05-study02-buyer-arms-disagree

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

7.2 Reserved seating and holds

Hold, expire, confirm, and re-sell. Guarantees held everywhere, but YugabyteDB refused valid confirmations until the refused statement was retried; and the funnel that would size the hold window is unanswerable because releasing an expired hold erases the evidence.

v2-10-guarded-confirm-refusals

normalized facts · single_run_directional

On YugabyteDB a guarded confirmation is not enough: correct designs refused valid holds, and retrying the refused statement once removed every refusal; on PostgreSQL the guarantee held everywhere. This claim is about refusals, not throughput, so no cost conclusion follows from it.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag

Source: studies/03-reserved-seating/reports/20260915T002411Z.md · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-gap-02-hold-funnel-unanswerable · gap · a schema limitation, not a missing measurement · full card in the evidence registry index

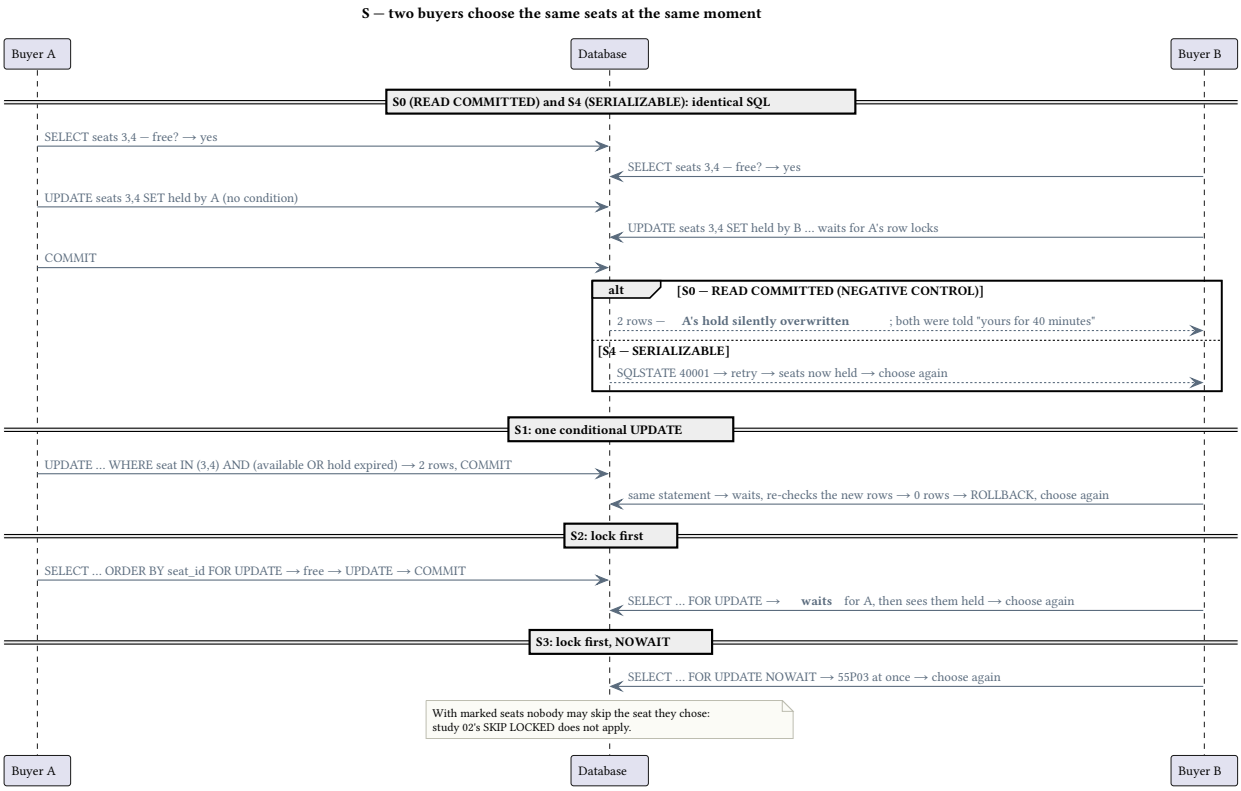


Figure 5: **Two buyers choose the same marked seats at the same moment.**

Form: sequence · Evidence: implemented design contract

> **Text equivalent.** Because a reserved-seating buyer may not be moved to another seat, Study 02's `SKIP LOCKED` scan does not apply here: the arbitration is a conditional update on the chosen seats (or a lock first, with the loser retrying) and one unguarded design is shown as a negative control. The figure is the contract the reserved-seating designs implement; it carries no measured value. Source: `studies/03-reserved-seating/diagrams/s_arbitration.puml`

7.3 Configuration portal

Read the overview often, replace configurations occasionally. A trigger-maintained rollup cost 4.0–5.7x on replacement across three runs; the application rollup kept writes near the reference. This is a small-scale result at one cardinality, and the trigger's own trials were imprecise.

v2-13-trigger-rollup-cost · run_replicated · PostgreSQL single node, small scale; three whole runs, three trials per cell in each; run-level replication only: separate runs on one machine, not hardware-independent, and still exposed to this host's measurement hazards; the trigger cell's own trials disagreed by 35–115% across the three runs, above the ~20% precision rule, so only the direction and an order-of-magnitude range are supported; the ratio is not a point estimate; the ~15x read gain belongs to the earlier single run and was not re-measured in the replication; the earlier single-run 6.85x figure did not reproduce and must not be quoted; the trigger's product is a structural guarantee, not speed; the three runs realized only two distinct design orders (seeds 202 and 303 coincided, and in the sibling read tranche all three seeds coincided); randomized order describes the method, and the realized order is recorded in each manifest; the penalty is size-dependent, not a constant: crossing the same pair with entries per installed product gives 2.07x at 10 entries, 4.38x at 60 and 24.20x at 240, so this ratio holds at the small scale default of 60 entries and must not be quoted as a property of the design without the size (v2–18-maintenance-size-interaction) · full card in the evidence registry index

7.4 Donor portal with an external cache

Serve warm profile reads from a cache. Throughput rose 2.2–3.5x and p99 fell an order of magnitude. Three instances sharing one relaxed cache recorded 87.81% wrong reads on the owned model and 87.24% on the legacy model: the publication must be fenced against the stale-fill race — a reader republishing S0 after a writer commits S1 and invalidates — not merely ordered after the commit. The single-instance relaxed cells recorded zero wrong reads, and the legacy process-local arms' zero was a bypass result (`v2-15`), so neither is evidence that an unfenced relaxed cache is safe.

v2-14-cache-throughput-gain · single_run_directional · one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); this

2.2–3.5x range is supported only by the PostgreSQL single-node + Redis cells; do not transfer the range to YugabyteDB, to a cluster, or to open-loop demand · full card in the evidence registry index

v2-15-three-instance-staleness · mechanism_supported · single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution · full card in the evidence registry index

v3-01-churn-crosses-real-ttl

external read copies / caches · single_run_directional

A sustained 1801 s churn phase against the fixed 300 s TTL crossed 6.0 real TTL boundaries with 0 wrong reads in 2,879,077 blended reads and 380,824 writes; the hard-expiry counter is 0 and only probabilistic early expiry fired (37,063), so an entry was never observed to be removed by its hard TTL. This is scoped mechanism and duration evidence, not a general cache result.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one PostgreSQL + Redis cell at scale small; one sustained measurement, no repeat; hard expiry fired zero times; that clause is an open gap, not a confirmation; closed loop, so read volumes are a floor; the 5x-for-120 s burst has no write-rate knob and was not run

Source: studies/05-cache-consistency/reports/20260923T1100Z-churn1800-through.md · run 20260923T1100Z-churn1800-through · tag run/05-cache-consistency/20260923T1100Z-churn1800-through

v3-03-equal-total-framing-labelled · single_run_directional · one run per framing, one host, scale small; the two arms are single trials; the value of the arm is the label and separation, not a throughput ratio; the difference is inside the run's spread and is not attributed · full card in the evidence registry index

7.5 Scenario transfers are labelled

Analogy — not measured here

A supermarket stock counter, an auction bid ledger or an e-commerce inventory pool is structurally similar to the ticketing hot row, but none of them was measured here. Use those transfers as analogies from the hot-row mechanism, never as separate evidence.

7.6 Where a scenario stops

Coverage gap

Two scenario families are explicitly incomplete, and the book does not fill them with words: the configuration-portal study did not build eight of its eighteen designs and did not measure cardinality or cadence, and the cache study still owes its hard TTL expiry, medium scale, tablet/leader placement and open-loop demand. The cache study **did** measure real TTL churn, in-run repeats and equal-total cache accounting on 2026-09-23, so those are no longer gaps. The vocabulary is worth fixing here: an **in-run repeat** is another measurement window inside one benchmark process, which reduces the scatter inside that process but is not a separate run of the whole experiment — where the cards say `Trials: 1`, they mean the experiment was run once and no more. None of the things listed above has been measured yet, which is not the same as having been measured and shown to make no difference: these are absences of evidence, not neutral or negative results.

v2-gap-03-study04-missing-designs

normalized facts · gap · coverage gap

Eight of the eighteen planned Study 04 configuration-portal designs are not implemented, and cardinality, cadence/churn, repeated randomized trials, equal-total resources, placement and YugabyteDB are unmeasured. These are coverage gaps, not null results.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: no conclusion may read 'the missing designs would not change this'; owned by task-20260922T025912Z-study04-v2-protocol

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v3-gap-01-study05-remaining-regimes

external read copies / caches · gap · partially superseded by v4-gap-01-physical-placement-unverified, v4-gap-02-endpoint-distribution-partial · coverage gap + instrument gap

Moved to successor claims: v4-gap-01-physical-placement-unverified, v4-gap-02-endpoint-distribution-partial.

Still open in this claim: hard-TTL expiry never observed removing an entry; medium scale beyond the harness maximum of 3,000 people; open-loop arrival demand.

Statement below is unchanged from its source registry; the moved dimensions are not restated as open here.

Study 05's remaining unmeasured regimes are explicit: the hard TTL never removed an entry in a measured cell (hard expiries 0 in all three churn cells), medium scale beyond the harness maximum of 3,000 people was not run, the tablet/leader placement distribution is absent, and open-loop arrival demand was not run.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: these are coverage gaps, not null results; medium scale is an order of magnitude beyond the current generator; placement verification needs an instrument the runner does not have; open-loop demand is owned by task-20260922T200831Z-study05-v2-open-loop

Source: studies/05-cache-consistency/reports/20260923T1100Z-churn1800-through.md · run 20260923T1100Z-churn1800-through · tag run/05-cache-consistency/20260923T1100Z-churn1800-through

8 Evidence and reproduction

CHAPTER · EVIDENCE AND REPRODUCTION

Every number in this book resolves to a claim in `book/evidence/v6/claims.json`, and every claim to something actually measured in a report this book cites. This chapter explains the confidence cards, the run-tag provenance model, and how to reproduce a run in Podman.

8.1 The provenance model

Direct evidence

A run tag marks the producing code state, not a tree that contains the run's results. Check out the tag for the code, then read the committed results directory for the data.

v2-gap-06-native-datastore-families

embedded documents · gap · coverage gap

No native document, key-value, graph or wide-column database was measured: JSONB/state is not evidence for a native document database, and Redis was measured only as a cache. Cross-family transfers to native datastores are analogy, not evidence.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: do not transfer the embedded-document or cache results to a native family

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap · unstable measurement

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

8.2 Reproducing the book itself

The book is built in Podman from a pinned toolchain; the manifest records every input and the resulting hash.

```
book/build.sh          # builds the pinned image, compiles, verifies, writes the
manifest
cat book/dist/build-manifest.json
```

8.3 Reproducing a measured run

```
git checkout run/<study>/<run-id>     # the producing code state
infra/<topology>.sh up                # take the benchmark lock first
cd studies/<study> && ./run-<engine>.sh
```

A run's results live in `studies/<study>/results/<run-id>/`; the report beside them is generated, never hand-edited, and its inputs digest identifies the data.

9 Part II — Concepts

The first part measured what each family does under this corpus's workloads. This part explains the mechanisms those results came from, in the order the decisions bind: where information is placed, what the indexes cost, who arbitrates a contended row, what derived state owes its maintainer, who owns expiry, what a cache must prove, and where a cluster actually puts the data. The last chapter is the checklist for reading any number in the first part.

Each concept names the claim cards that measured it, and the glossary at the end of the book is the single definition site for every term used here.

10 Information placement

CONCEPT · INFORMATION PLACEMENT

Placement is the decision the rest of the book hangs on: which single fact owns which question, and what is allowed to exist only because it answers a question faster. Every design in the corpus is a different answer to "where does this fact live, and who maintains it".

10.1 The ladder

- **Derive it.** Answer the question from normalized facts at read time. Nothing to maintain, nothing to go stale; the cost is paid per read.
- **Index an existing fact.** Add the structure that turns a scan into a lookup. The cheapest derived object, but it taxes every write and only helps questions the schema can already express.
- **Materialize it.** Store the aggregate on the parent. Big read wins, a maintenance cost on every child write, and a correctness question whenever a write path forgets it.
- **Copy it down (rolldown).** Put parent information on the child to avoid a join. Measured as a package with its indexes, never as a bare column.
- **Copy it out (cache).** Serve the answer from beside the database, outside the authoritative transaction. It can be the lowest-latency hot read for the measured cacheable workload, and it is the step that crosses the transaction boundary — so the design acquires a separate freshness and failure-domain contract. Indexes, rollups and rolldowns carry their own maintenance correctness; what is special here is where the copy lives, not that it is copied.

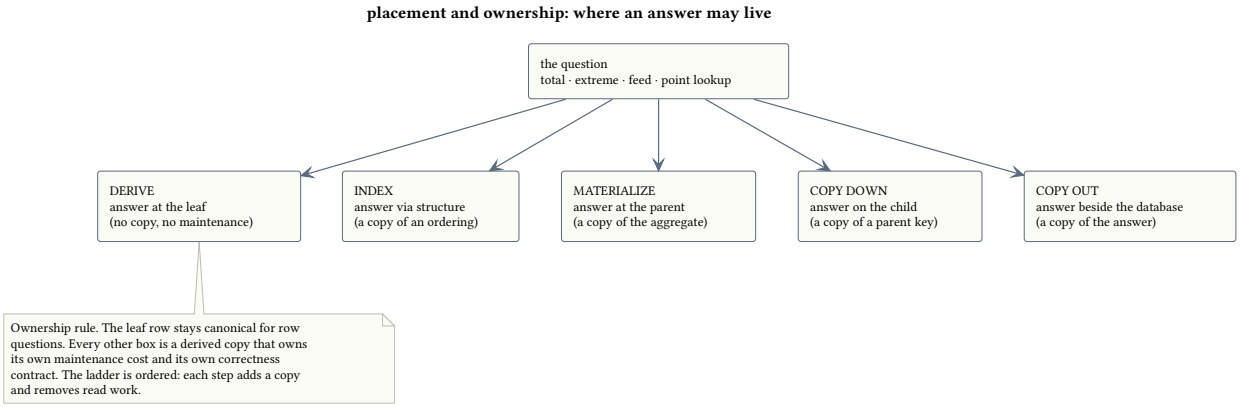


Figure 6: **Placement and ownership: where an answer may live.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** The leaf row stays canonical for row questions. **Derive stores no additional copy.** The remaining rungs introduce some stored derived structure or copy, each owning its own maintenance cost and its own correctness contract: **index** adds a copy of an ordering, **materialize** copies the aggregate upward, **copy down** copies a parent key onto the child, and **copy out** serves the answer beside the database under a freshness contract. The ladder is ordered by how much read work each stored structure removes. The figure is a map of the corpus's decisions and carries no measured value.

Direct evidence

Ask what the schema must be able to **answer** before asking how fast it answers. Two measured answerability failures make this concrete: refunds were unanswerable in 14 of 15 ticketing designs, and no reserved-seating design could size a hold window because releasing an expired hold erases the evidence that it existed.

v2-07-refund-answerability-and-storage · single_run_directional · answerability is a schema fact, not a timing; the +33–35% storage delta is a relation-size difference measured in the primary run and deterministic for the same dataset and design; it is not a repeated timed result; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded · full card in the evidence registry index

v2-gap-02-hold-funnel-unanswerable · gap · a schema limitation, not a missing measurement · full card in the evidence registry index

10.2 Boundaries

Coverage gap

Placement says nothing about **physical** colocation. The corpus does not verify where a cluster put its data, so a placement claim here is a design claim, not a topology result.

10.3 The checklist

The eight questions, answered for this concept; the status in brackets is the evidence behind the answer.

- **Invariant.** After every acknowledged mutation, a derived answer equals what the base facts imply. [direct]
- **Where the information lives.** In the base table, in a copied parent key on the child, in an aggregate on the parent, or inside an embedded child document. [direct]
- **Where the contention is.** The parent row that every child write touches once an aggregate or a document lives there. [direct]
- **What work moved from reads to writes.** Aggregation (rollup) and child retrieval (embedding) both move to the write path. [direct]
- **What state is derived.** Parent aggregates, copied parent keys, and embedded child arrays. [direct]
- **What can become stale.** An application-maintained rollup published outside the mutation's transaction, and an embedded slice that lags its table. [mechanism]
- **Where it stops working.** Embedding, once a parent's history grows without bound; a rollup, once its maintenance cost exceeds the read gain it buys. [gap]
- **The experiment that would settle it.** Isolated pairs at matched SQL, swept over cardinality and history so the crossing is visible. [direct]

11 Indexes and hot rows

CONCEPT · INDEXES AND HOT ROWS

Two different tools get confused because both look like “make the query fast”. An index changes how many rows the engine touches; a hot-row decision changes who may touch one row at the same time.

two different tools: the index narrows the scan, the hot row serialises the writers

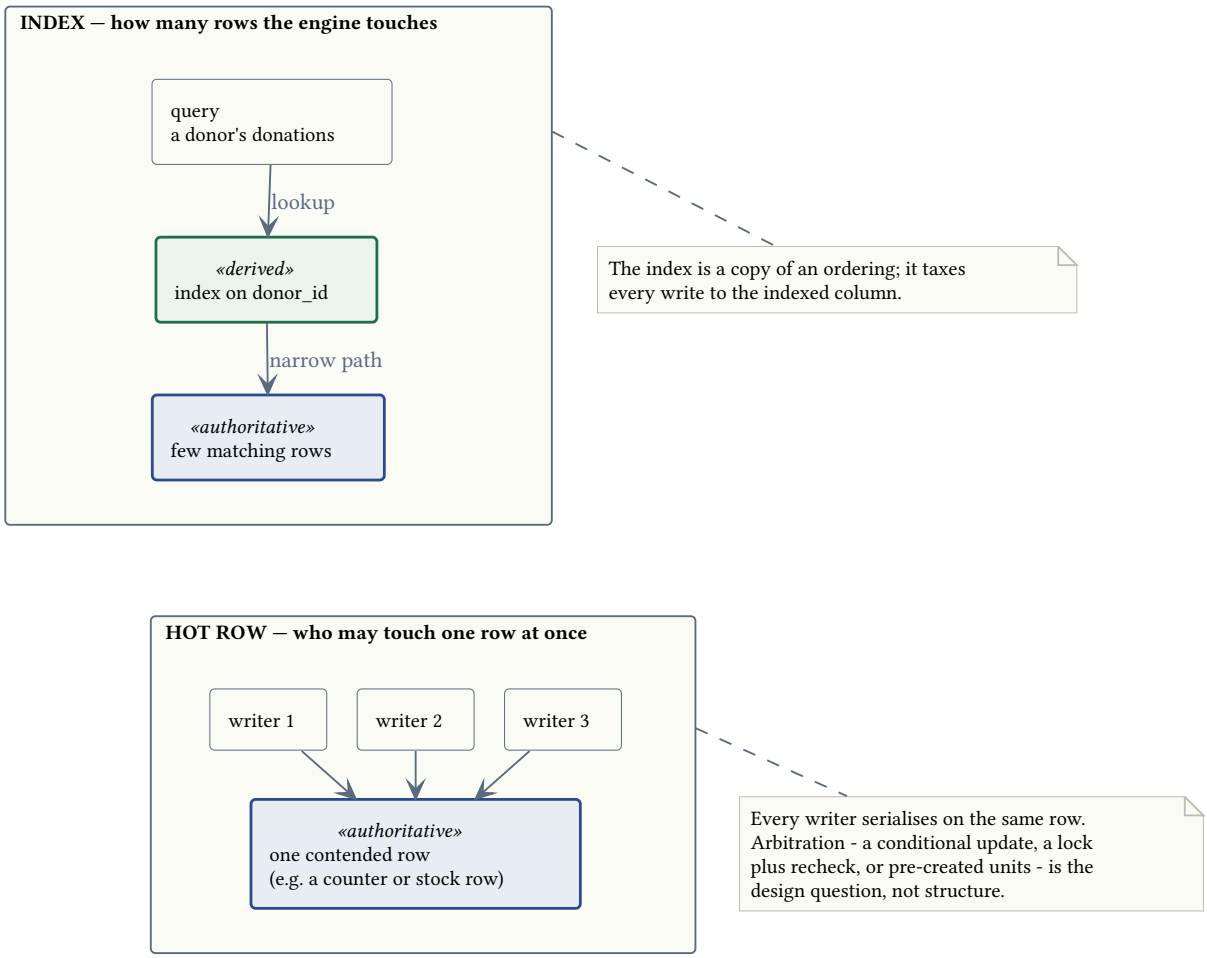


Figure 7: **An index narrows the scan; a hot row serialises the writers.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** Left, the index panel: a query follows a narrow index path to a small subset of rows, so the engine touches less — the index is a copy of an ordering and taxes every write to the indexed column. Right, the hot-row panel: several writers converge on one contended row, so they serialise and the design question is arbitration, not structure. The figure separates the two tools and asserts no rate.

11.1 Indexes

An index answers a question the schema can already express, at the cost of every write to the indexed column. In the charity tree the first index was the first large read win, and flattening the foreign key (a rolldown package) bought more still.

11.2 Hot rows

When one row is the contention point — one event’s ticket count, one product’s stock — the design question is arbitration, not structure. Pre-created seats taken by compare-and-set or `SKIP LOCKED` sold 4–8x faster than any single counter row; the counter also blocks unrelated edits to the same row, which is a correctness-adjacent cost the throughput number hides.

11.3 Direct evidence

v2-01-normalized-index-first · single_run_directional · PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query · full card in the evidence registry index

v2-06-hot-drop-designs · single_run_directional · one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only · full card in the evidence registry index

11.4 Boundaries

The hot-row result is one workload at small scale on one host. Read it as "the counter row serialises everything", not as a capacity figure.

11.5 The checklist

The eight questions, answered for this concept; the status in brackets is the evidence behind the answer.

- **Invariant.** The answer must not change because an access path changed; an index may only change how the answer is found. [direct]
- **Where the information lives.** In the table. An index locates rows; it does not hold the answer. [direct]
- **Where the contention is.** The single hot row that a counter or an aggregate lives on — one event row, one parent. [direct]
- **What work moved from reads to writes.** Maintaining an index, or a counter, is write work bought to make reads cheap. [mechanism]
- **What state is derived.** Nothing, for an index. A counter or rollup column is derived, and belongs to the derived-state concept. [mechanism]
- **What can become stale.** A rollup column can disagree with its entries. An index cannot be stale, only unused. [mechanism]
- **Where it stops working.** On a table small enough that the whole relation is cached, an index stops paying — measured only as a control that could not fire at 3,600 rows. [mechanism]
- **The experiment that would settle it.** An identical-SQL index-versus-no-index pair, plus a writer sweep to show where the hot row takes over. [direct]

12 Concurrency control

CONCEPT · CONCURRENCY CONTROL

Two strategies at minimum: check a version and retry, or take a lock before updating. Both protect the same invariant; they differ in where contention is paid and what the failure looks like.

12.1 Optimistic (version-checked)

The writer reads, checks that the version is unchanged, writes conditionally, and retries on a mismatch. Under low contention it is close to free; under high contention retries become the workload. The measured upper bound put lock-before-update at 1.76x the optimistic design under one hot key and 16 writers, with the caveat that the run could not separate the lock's advantage from extra per-retry harness work.

12.2 Pessimistic (lock-before-update)

The writer waits for the row, then updates. With pessimistic locking, contention is primarily paid as waiting/blocking rather than optimistic conflict retries; depending on the workload that may increase **or** decrease observed latency. Application-level retries can still be required — for deadlocks, lock timeouts, serialization failures or other transient failures — so the retry loop does not universally disappear. On a single engine primitive, `SKIP LOCKED` and compare-and-set both avoid the counter-row bottleneck measured in ticketing.

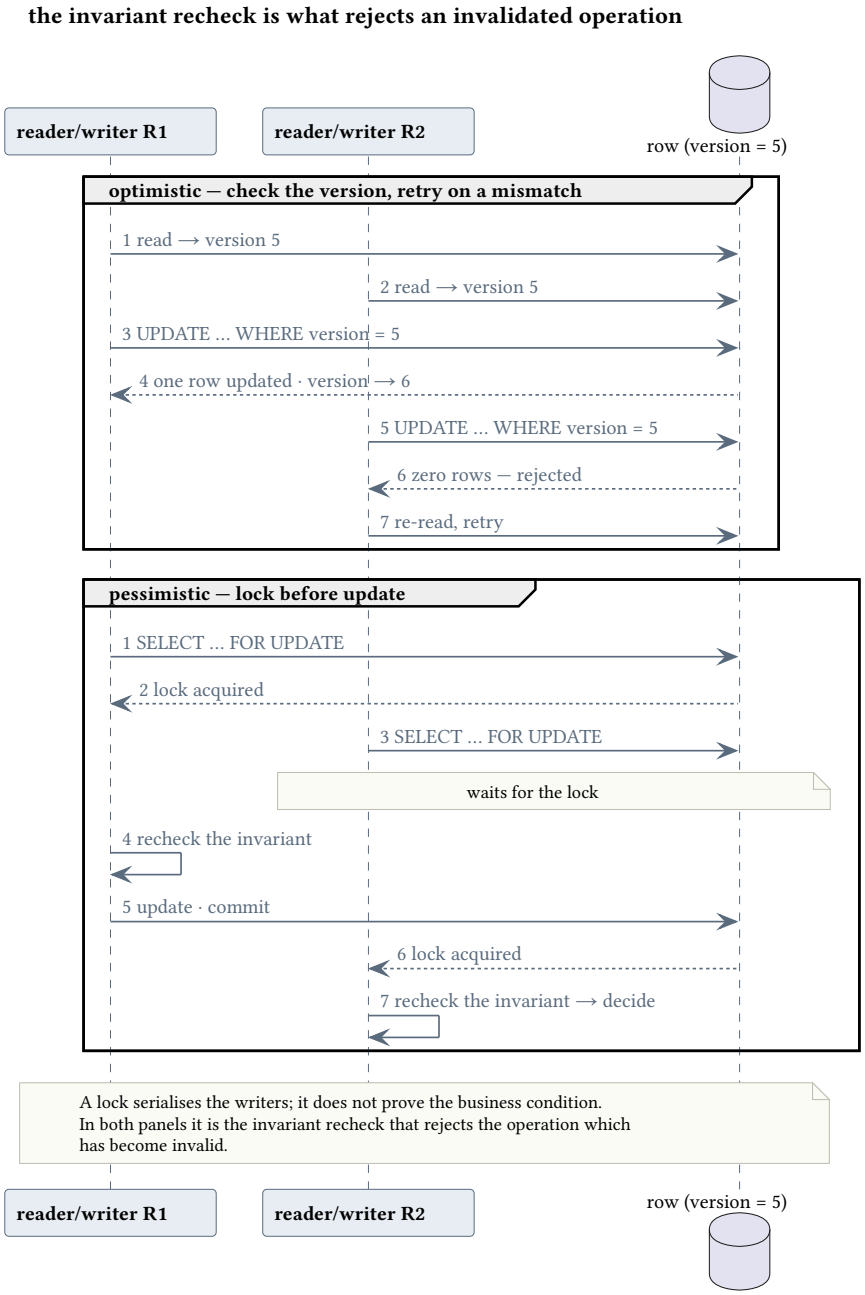


Figure 8: **The invariant recheck, not the lock, rejects an operation that has become invalid.**

Form: sequence · Evidence: conceptual illustration

> **Text equivalent.** Two parallel sequences over one row at version 5. **Optimistic**: both writers read version 5; R1′s `UPDATE … WHERE version = 5` updates one row and moves the version to 6, so R2′s conditional update touches zero rows and R2 re-reads and retries. **Pessimistic**: R1 takes `SELECT … FOR UPDATE`, R2 waits, R1 rechecks the invariant, updates and commits, then R2 acquires the lock and rechecks the invariant before deciding. In both panels the lock or the version check serialises the writers, and the **invariant recheck** is what rejects the operation that has become invalid. The figure draws `v2-12`′s mechanism and asserts no rate.

12.3 The negative control matters here

An unchecked read-modify-write lost 93.7% of acknowledged updates in the same run that measured the lock advantage. A concurrency result without its lost-update control is not evidence.

v2-12-lock-vs-version-upper-bound · single_run_directional · one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve · full card in the evidence registry index

12.4 Where the two designs cross

The lock is not simply faster. Swept across concurrent writers on one hot key, the pessimistic lock gives up 11% of the version check's throughput at one writer and leads from two writers on, reaching 1.68x at 32. The movement is on the optimistic side: its throughput falls 2.9x across the sweep while its conflict count rises from 0 to 21,301. That correlation is not an explanation, and the obvious one was tested and rejected — with the retry budget set to zero the ratio at 16 writers does not narrow (1.60 against 1.61), and the design that never retries moves by the same amount as the one that does. So "which is faster" has no answer without the concurrency level, and the interval where the ordering reverses is what a recommendation should name.

v2-17-optimistic-pessimistic-crossover

normalized facts · single_run_directional

Under one hot key, a bounded-retry version check (c1_optimistic_version) and a lock-before-update (c2_pessimistic_lock) cross over between one and two concurrent writers: the lock delivers 0.89x the version check's throughput at 1 writer (1,177 against 1,329 ops/s) and 1.10x at 2 writers (1,285 against 1,167), rising to 1.68x at 32 writers (877 against 523) and 1.60x at 64, while the version check's own throughput falls 2.9x across the sweep (1,329 to 466 ops/s) and its conflict count rises from 0 to 21,301.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale, one hot key, one measurement per level across seven runs (1, 2, 4, 8, 16, 32 and 64 writers), so the curve carries no spread at any level; the retry budget was tested as the mechanism and rejected: with -retries 0 the c2/c1 ratio at 16 writers is 1.60 against 1.61 with retries, and c2_pessimistic_lock - which never retries - moved by the same 10% as c1 between the two same-day arms, so the lock's advantage is not attributable to retry work (analysis 20260926T-retry-control, runs 20260926T142926Z and 20260926T143014Z); the seven runs used two phase sets (the first four also ran read and write phases); the contention metric should not depend on that, but it is an assumption rather than a measurement; no negative control in this sweep; control_coverage is recorded absent rather than implied; the 1-writer point is the edge of the sweep, so the optimistic side is bracketed by one level only; absolute numbers are a laptop comparison; the shape of the curve is what transfers; absolute throughput is day-specific: c2 measured 869 ops/s on 2026–09–23 and 1,056 on 2026–09–26 at identical settings, phases and writer count, so only the within-run ratio is carried across days

Source: studies/04-configuration-portal/reports/20260923T0011Z-ctlC-w1.md · run 20260923T0011Z-ctlC-w1 · tag run/04-configuration-portal/20260923T0011Z-ctlC-w1

12.5 Boundaries

One key, one engine, one trial, small scale. The ratio is an upper bound and a point, not a curve.

12.6 The checklist

The eight questions, answered for this concept; the status in brackets is the evidence behind the answer.

- **Invariant.** The business rule holds under concurrent writers: no lost increment, no oversold seat, no double sale. [direct]
- **Where the information lives.** In the current row the writers contend for, plus whatever version or lock column guards it. [direct]
- **Where the contention is.** One hot key or row; the corpus measures one event or one installed product. [direct]
- **What work moved from reads to writes.** Conflict resolution itself: a version check or a lock turns a read-then-decide into one guarded write. [mechanism]
- **What state is derived.** Nothing in this concept; a counter or rollup a concurrent writer maintains is derived state, which is its own concept. [mechanism]
- **What can become stale.** The value a writer read before losing the race — which is why the guard has to be part of the write. [mechanism]
- **Where it stops working.** The ordering is workload-dependent: at one writer the version check leads, and from four writers the lock leads by 1.26–1.60x. [mechanism]
- **The experiment that would settle it.** A writer-count sweep of the same pair with retries disabled, so the lock's advantage is separated from per-retry work. [direct]

13 Derived state and history

CONCEPT · DERIVED STATE AND HISTORY

Two different things are stored beside the canonical row, and conflating them produces the wrong design. **Derived state** — a rollup, a rolldown, an external read copy — optimizes an answer from current state and owes its maintainer a correctness contract on every contributing write. **Retained history** — an event or ledger table, an audit trail, a temporal history, snapshots — preserves information that would otherwise disappear. Derived state can be rebuilt from the facts; retained history cannot be rebuilt from a current-state-only representation.

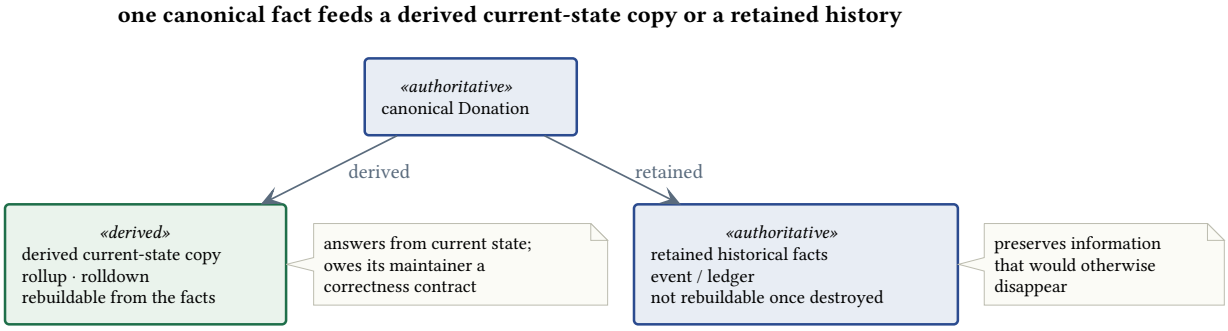


Figure 9: **One canonical fact feeds a derived current-state copy or a retained history.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** A canonical fact has two stored branches. A **derived current-state copy** (rollup or rolldown) answers from current state, can be rebuilt from the facts and owes its maintainer a correctness contract on every contributing write. **Retained historical facts** (an event/ledger) preserve information that would otherwise disappear and cannot be rebuilt once a current-state-only representation has destroyed it. The figure asserts no rate.

13.1 Maintenance is the design

A rollup can be maintained by the application or by a database trigger. The trigger makes the guarantee structural; the measured cost was 4.0–5.7x on whole-configuration replacement across three runs (median 4.94x), with a 22–46x worse latency tail, while the application rollup kept writes near the reference. That ratio is a reading at one size, not a property of the design: crossed with the size of a replacement it runs 2.07x at 10 entries per installation, 4.38x at 60 and 24.20x at 240, because the trigger does its work once per row written while the application does one statement per replacement. Neither is universally right: choose who must never forget, and then ask how large a replacement is.

v2-13-trigger-rollup-cost · run_replicated · PostgreSQL single node, small scale; three whole runs, three trials per cell in each; run-level replication only: separate runs on one machine, not hardware-independent, and still exposed to this host's measurement hazards; the trigger cell's own trials disagreed by 35–115% across the three runs, above the ~20% precision rule, so only the direction and an order-of-magnitude range are supported; the ratio is not a point estimate; the ~15x read gain belongs to the earlier single run and was not re-measured in the replication; the earlier single-run 6.85x figure did not reproduce and must not be quoted; the trigger's product is a structural guarantee, not speed; the three runs realized only two distinct design orders (seeds 202 and 303 coincided, and in the sibling read tranche all three seeds coincided); randomized order describes the method, and the realized order is recorded in each manifest; the penalty is size-dependent, not a constant: crossing the same pair with entries per installed product gives 2.07x at 10 entries, 4.38x at 60 and 24.20x at 240, so this ratio holds at the small scale default of 60 entries and must not be quoted as a property of the design without the size (v2–18-maintenance-size-interaction) · full card in the evidence registry index

v2-18-maintenance-size-interaction

rollups / materialized aggregates · single_run_directional

Crossing who maintains the parent rollup (trigger n3_rollup_trigger against application n4_rollup_app) with the size of a replacement (10, 60 and 240 entries per installed product) shows the maintenance penalty is a function of size rather than a constant: the application design delivers 2.07x the trigger design on whole-configuration replacement at 10 entries (952 against 459 ops/s), 4.38x at 60 (631 against 144) and 24.20x at 240 (242 against 10), while the trigger's p99 rises from 3 to 156 ms against the application's 1 to 8 ms and no rollup mismatch is recorded at any size.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale, one measurement per tier across 10, 60 and 240 entries per installed product, so the curve carries no spread at any level; one operation only: w05_replace_all replaces the whole configuration, and a workload that appends a single entry would sit at the low end of this curve at any size; the axis is capped by the small scale at 240 entries and the curve is still rising steeply there, so nothing here bounds where it flattens; no negative control in these three runs; control_coverage is recorded absent rather than implied; two designs cannot separate the two mechanisms that predict this shape: the trigger fires once per row written, and its per-row work may also be heavier than the application's single statement; the tier-60 cell reads 4.38x, inside the replicated range v2–13 measured for the same operation at the same tier on another day (3.99–5.69x); that cross-day comparison carries the approximately 30 percent day term the retry control measured

Source: studies/04-configuration-portal/reports/20260926T152621Z.md · run 20260926T152621Z · tag run/04-configuration-portal/20260926T152621Z

13.2 History is answerability

Retained history is not faster; it is the difference between being able to answer a question after the event was undone and not. A current-state-only representation that overwrites or deletes the earlier fact cannot reconstruct it, whichever physical structure stores the answer — an event table, an audit trail, a temporal history or a snapshot can each supply it. The refunds window was unanswerable in 14 of 15 ticketing designs and cost about 33–35% more storage with a ledger. That is a correctness purchase, priced honestly. Retaining history does not itself fence concurrent writers or order events; that is a separate, explicit decision.

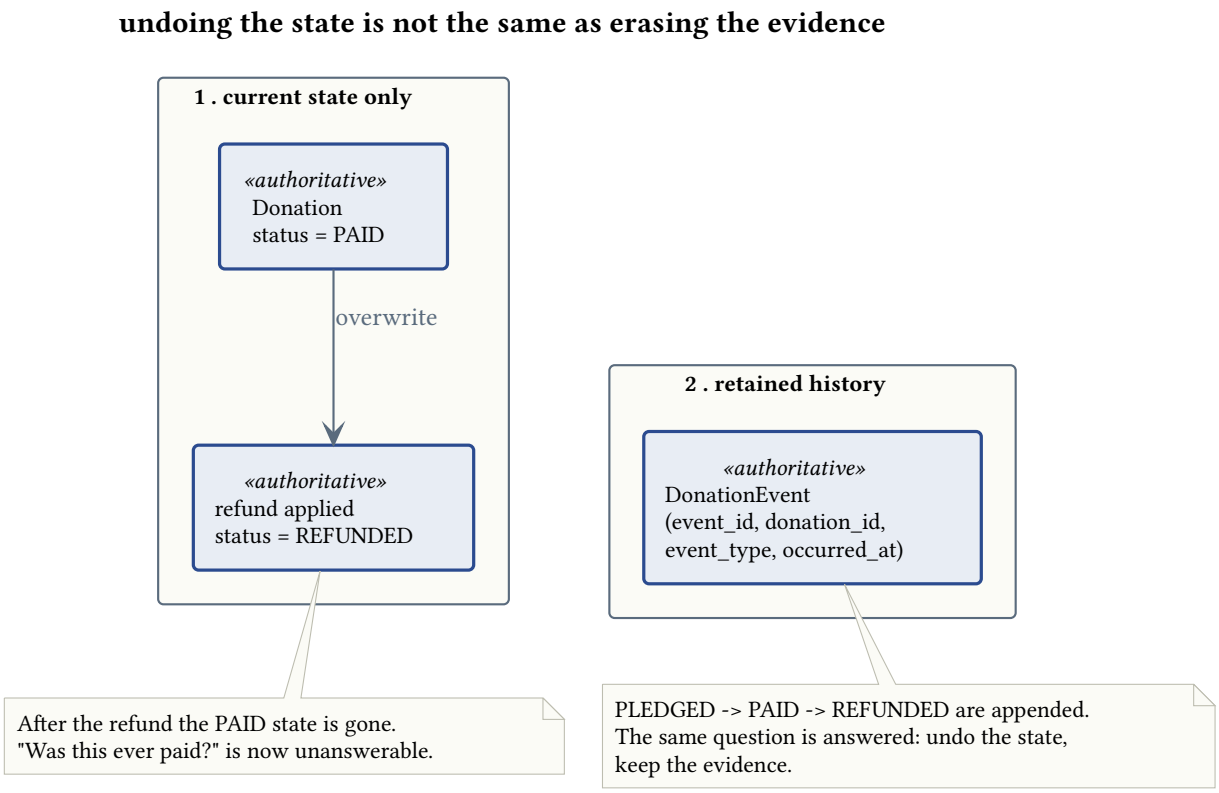


Figure 10: **Undoing the state is not the same as erasing the evidence.**

Form: structure · Evidence: conceptual illustration

Text equivalent. Left, a current-state-only representation overwrites a Donation's PAID status on refund, so "was this ever paid?" becomes unanswerable. Right, appended `DonationEvent` rows (PLEDGED, PAID, REFUNDED) answer the same question: undoing the state keeps the evidence. The figure is the difference between derived state and retained history, and it asserts no rate.

13.3 Direct evidence

v2-13-trigger-rollup-cost · run_replicated · PostgreSQL single node, small scale; three whole runs, three trials per cell in each; run-level replication only: separate runs on one machine, not hardware-independent, and still exposed to this host's measurement hazards; the trigger cell's own trials disagreed by 35–115% across the three runs, above the ~20% precision rule, so only the direction and an order-of-magnitude range are supported; the ratio is not a point estimate; the ~15x read gain belongs to the earlier single run and

was not re-measured in the replication; the earlier single-run 6.85x figure did not reproduce and must not be quoted; the trigger's product is a structural guarantee, not speed; the three runs realized only two distinct design orders (seeds 202 and 303 coincided, and in the sibling read tranche all three seeds coincided); randomized order describes the method, and the realized order is recorded in each manifest; the penalty is size-dependent, not a constant: crossing the same pair with entries per installed product gives 2.07x at 10 entries, 4.38x at 60 and 24.20x at 240, so this ratio holds at the small scale default of 60 entries and must not be quoted as a property of the design without the size (v2–18-maintenance-size-interaction) · full card in the evidence registry index

v2-09-postgres-ledger-ratio-unsettled · single_run_directional · buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted · full card in the evidence registry index

13.4 Boundaries

The ledger's write cost depends on the engine and on the order the designs were run at the small size, so quote the run's own range and the disagreement rather than a single headline multiplier. The load-time multiplier is not published at all.

13.5 The checklist

The eight questions, answered for this concept; the status in brackets is the evidence behind the answer.

- **Invariant.** Published derived state agrees with the entries it summarises, and retained history is never erased by the mutation that follows it. [direct]
- **Where the information lives.** In the base entries, in an aggregate on the parent, and — for answerability — in an append-only record that the mutation does not overwrite. [direct]
- **Where the contention is.** The parent row an aggregate is maintained on, which every child write touches. [direct]
- **What work moved from reads to writes.** Aggregate maintenance. History is different: it is write-only work bought for the questions it lets you answer later. [direct]
- **What state is derived.** The rollup columns. History is a fact, not a derived value, which is why it is not "stale" when it disagrees with current state. [direct]
- **What can become stale.** Any aggregate published after the commit that produced it — the drift control demonstrates it. [mechanism]
- **Where it stops working.** A per-row trigger's cost grows with the rows a replacement touches — now measured at three sizes (2.07x, 4.38x, 24.20x at 10, 60 and 240 entries) rather than stated as a tendency; history costs storage (33–35% for the ledger). [direct]
- **The experiment that would settle it.** The n3/n4/x2 isolation pair, swept over entries per installation to find where the two maintenance strategies cross. [direct]

14 Expiry and clock authority

CONCEPT · EXPIRY AND CLOCK AUTHORITY

Expiry is a write that no client issues. That makes it the place where “whose clock is right” stops being philosophical: the sweeper, the application clock and the database all disagree eventually, and the design must say who is allowed to decide.

14.1 What the corpus shows

YugabyteDB refused valid hold confirmations under a guarded design, and retrying the refused statement once removed every refusal at no measurable cost. The practical lesson is smaller than it sounds: an expiry design needs a retry story, because the mechanism can refuse a correct write.

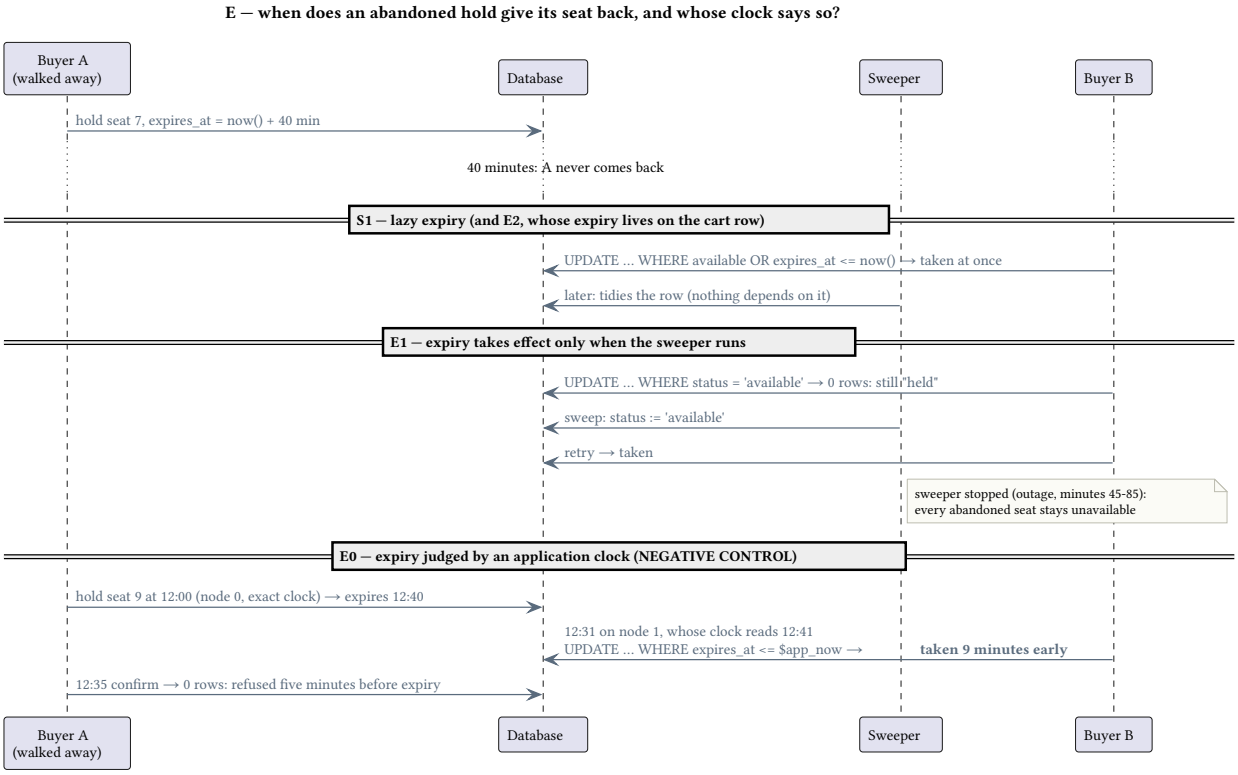


Figure 11: **When an abandoned hold gives its seat back, and whose clock says so.**

Form: sequence · Evidence: implemented design contract

> **Text equivalent.** The figure lays out the expiry designs side by side — lazy expiry judged by the database clock, a sweeper that decides when a seat returns, and an application clock shown as a negative control that can release a seat early. It is the contract the reserved-seating designs implement and carries no measured value; the funnel that would size a hold window remains unanswerable (`v2-gap-02`). Source: `studies/03-reserved-seating/diagrams/e_expiry.puml`

14.2 What it cannot yet show

No design can answer the funnel that would size a hold window. Releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat that was never held. The missing evidence is a schema property, not a missing measurement.

v2-10-guarded-confirm-refusals · single_run_directional · v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag · full card in the evidence registry index

v2-gap-02-hold-funnel-unanswerable · gap · a schema limitation, not a missing measurement · full card in the evidence registry index

14.3 Boundaries

Do not transfer a single-engine expiry result to a clock-synchronised cluster without a placement and clock test; the corpus has neither.

14.4 The checklist

The eight questions, answered for this concept; the status in brackets is the evidence behind the answer.

- **Invariant.** A hold made while it is valid is honoured, nothing is granted or sold after expiry, and no seat leaks. [direct]
- **Where the information lives.** In `expires_at` on the held row — or, in one variant, once on a cart row rather than copied to every seat. [direct]
- **Where the contention is.** The seat rows of one chosen block, which cannot be skipped the way study 02′s interchangeable seats can. [mechanism]
- **What work moved from reads to writes.** Expiry checks move to the moment of use, and a sweeper does the work at rest instead of on the buyer's path. [mechanism]
- **What state is derived.** Availability is derived, not stored: with lazy expiry an expiry changes no row, so no exact availability counter can be maintained. [mechanism]
- **What can become stale.** Any judgement of expiry made from an application clock, and any cached availability figure. [direct]
- **Where it stops working.** When the clock is not the database's — the injected-skew control fired — and when the sweeper stalls, which leaks seats. [gap]
- **The experiment that would settle it.** An injected application-clock skew and a sweeper outage, beside the honoured-hold invariant. [direct]

15 Cache consistency

CONCEPT · CACHE CONSISTENCY

A cache is a read copy with a freshness contract. The design question is not “which cache pattern?” but “what does a hit **prove**, and who can observe every change?”.

Invalidation and publication fencing are different operations. Invalidation removes or marks an existing cache entry stale. It cannot stop a reader that loaded an older value from publishing it afterwards — which is exactly the race the stale-fill figure below draws. A **publication fence** is the additional check that refuses that fill when the source moved after the reader took its snapshot; its strongest form is **source-generation validation**, where the filler checks the authoritative current generation against the one it observed before publishing. A **sink-side fencing token** is a different mechanism: the protected sink remembers the highest token it has accepted and rejects an older one, so it orders operations only once a newer token has actually reached the sink. Neither is **cache CAS**, which conditionally modifies cache state against an explicitly stated expected generation, and neither is **hit validation**, which checks a served value against authoritative state at the hit boundary. Writing “invalidation is a fence” makes the fence sound like a deletion, and the deletion is the part that does not close the race. This is the concept the external-read-copies family exists to hold.

source correctness belongs to the mutation; copy correctness belongs to the publication

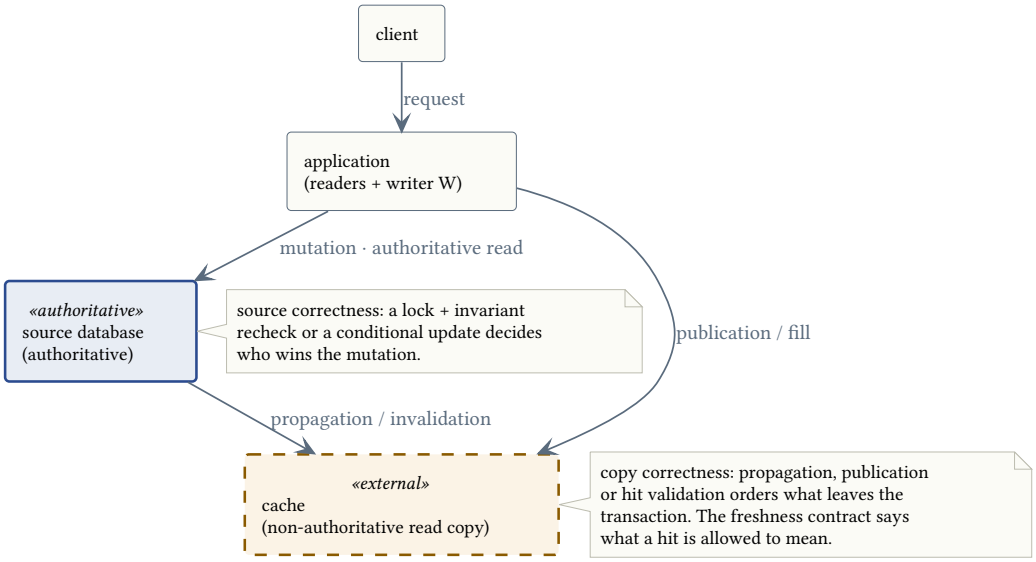


Figure 12: **The cache is a non-authoritative read copy beside the authoritative database.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** A client reaches the application, which talks to the authoritative source database and to a cache. Mutation and source reads belong to the database; **source correctness** is decided by the mutation itself (a lock plus invariant recheck, or a conditional update). The cache holds a **non-authoritative read copy**: **copy correctness** is decided by propagation, publication or hit validation, and the declared freshness contract says what a hit is allowed to mean. The figure is the architecture the races below take place in, and it asserts no rate.

15.1 The flow: contract first, pattern last

- **1. Name the contract.** *Strict after acknowledgement*: a read that starts after a write to the same key is acknowledged must not return an older committed value. Or a **bounded-staleness** budget, or **session / read-your-writes**, or best effort. State which one, on the page.
- **2. Establish change-observation completeness.** Can every mutation participate — bulk jobs, migrations, deletes, recreates, external writers? If not, a strict **cache-hit** proof is unavailable: use an authoritative read, a complete change source, or downgrade the contract explicitly.

- **3. Identify the source capabilities.** A frozen schema still supports service-side generations and authoritative reads; an owned schema can add a monotonic version, a transactional outbox and a deletion generation. These improve proof and recovery; they do not confer freshness.
- **4. Choose topology and the publication protocol.** Process-local versus shared decides the coordination domain; cache-aside versus write-through decides who fills or republishes. State the ordering proof — one of the named mechanisms below.
- **5. Add performance controls separately.** A bounded fill lease suppresses duplicate fills while it is held; a hard TTL bounds how long an entry is **eligible to serve** and probabilistic early expiry spreads the refill work. None of these supplies currentness.

Forbidden is separate from stale. A value that is impossible or was never committed is a defect under every contract, not a relaxation of one. **Relaxed** is a family of weakenings rather than one contract, so which member is in force has to be named:

- **Strict after acknowledgement** — the rule the comparison in this chapter is made against.
- **Session / read-your-writes** — a client can see its own committed writes.
- **Bounded staleness** ($\leq \Delta$) — a hit may lag the latest commit by at most a declared Δ.
- **Eventual freshness** — convergence is required; no distance or deadline is declared.
- **Best effort** — nothing beyond a valid committed value is promised.

None of these permits a dirty, torn or uncommitted value, and no configuration measured here produced an impossible value — measuring that is how the sentence is checked rather than assumed. The relaxed contracts this study measured are unbounded-relaxed: compared with the strict rule below, they allow a committed but older value, without declaring how old it may be. They are therefore not bounded-staleness contracts, and counting how often they break the strict rule says nothing about whether they stay inside a Δ window.

15.2 The strict contract, written down

Strict after acknowledgement is the rule every configuration measured here is judged against, so it is worth stating formally rather than in prose. Let `W1` be a write to a key, `ack(W1)` the moment its acknowledgement is observed, `R1` a read of the same key, and `begin(R1)` the moment it starts.

`begin(R1) > ack(W1)` $\Rightarrow$ `R1` must not return a version older than `W1`
unless a later write to the key supersedes `W1`

Every other contract is a weakening of that line, and the weakening must be named: a **bounded-staleness** budget, **session / read-your-writes**, or **best effort**. That line is the yardstick for the counts in this chapter — a "wrong read" is one that breaks it, not one that returns something impossible.

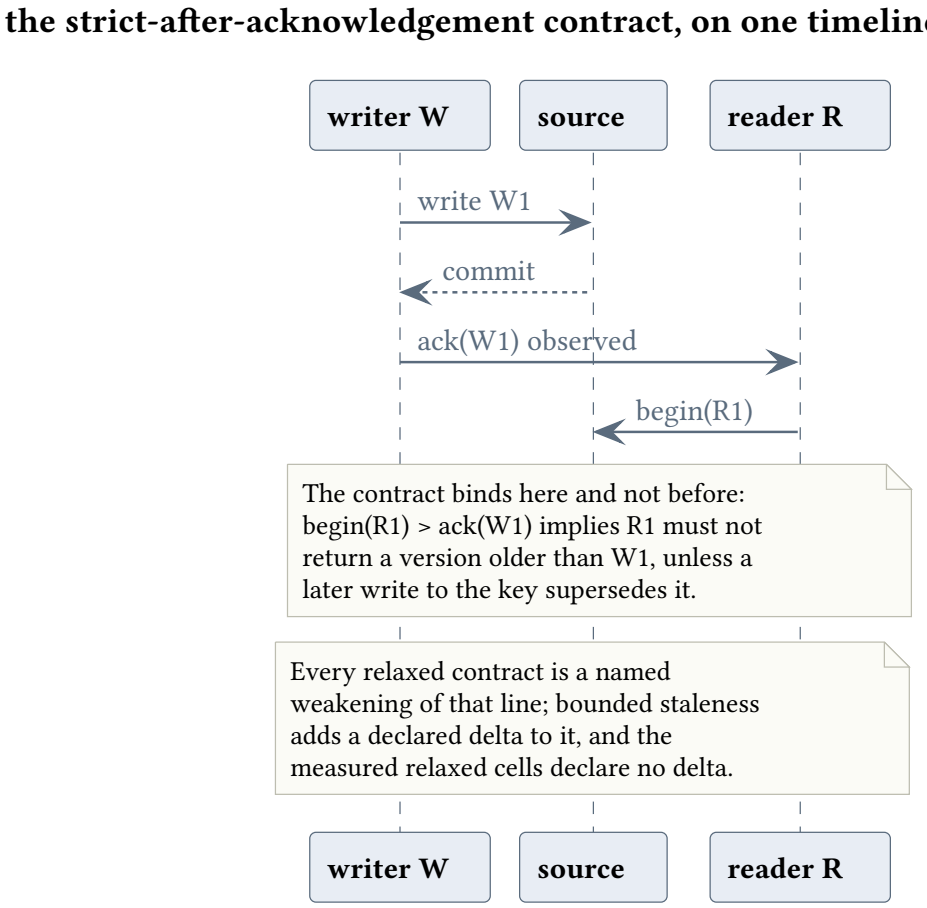


Figure 13: **The strict-after-acknowledgement contract, and the point on the timeline where it binds.**

Form: timeline · Evidence: conceptual illustration

> **Text equivalent.** A writer commits `W1`, the acknowledgement is observed, and a later read begins. The contract binds at `begin(R1) > ack(W1)`: the read must not return a version older than `W1` unless a later write supersedes it. Everything the book calls relaxed is a named weakening of that line, and a bounded-staleness contract adds a declared delta the measured cells do not declare. The figure draws the contract; the rates beside it are the registry's.

15.3 The named mechanisms, one primary failure mode each

A mechanism is not a synonym for freshness. Each one addresses one primary failure mode or control objective, and the "does not solve" line matters as much as the guarantee: several of these can be present and the contract still be broken.

Source correctness — arbitrating the mutation itself

These protect the authoritative row. They decide **who wins** a contended update; they say nothing about a copy that has already left the transaction.

Pessimistic lock + invariant recheck

Race closed	two database writers updating the same row
Mechanism	`SELECT … FOR UPDATE` serialises the writers, and the writer re-reads the row under the lock and rechecks the transition's precondition before applying it
Guarantee	serialisation: the two writers do not interleave. Rejecting the obsolete operation is the **recheck's** contribution, not the lock's — a lock alone guarantees neither
Does not solve	a stale read in another connection, a lost write outside the lock's scope, or any copy of the row

Evidence: active registered claim — `v2-12`

Conditional update / compare-and-set

Race closed	an update applied to a row whose state has already moved on
Mechanism	the expected state or version goes into the write itself (`WHERE … AND status = …`); a zero-row update reports that another writer got there first
Guarantee	the transition applies only while the row still satisfies the expectation, so the loser is rejected rather than merely serialised
Does not solve	the retry cost the loser now pays, and any copy of the row

Evidence: active registered claim — `v2-06`

Copy correctness — ordering what leaves the transaction

These protect the cache. One of them closes the stale-fill race; two of them — the fill lease and the TTL — are load controls that prove nothing about freshness.

Invalidation

Race closed	an entry that is still resident after its source changed
Mechanism	the writer removes or marks the entry stale after committing
Guarantee	once the invalidation has been applied, subsequent lookups of that cache entry miss or see it marked stale — call it what it is: a deletion with a message attached. Reads and fills already in flight are outside this guarantee
Does not solve	the stale-fill race. A reader that snapshotted S0 before the invalidation can still publish S0 afterwards; deletion orders nothing about a fill already in flight

Evidence: used by every write-through configuration measured here; on its own it never made a shared relaxed cache strict

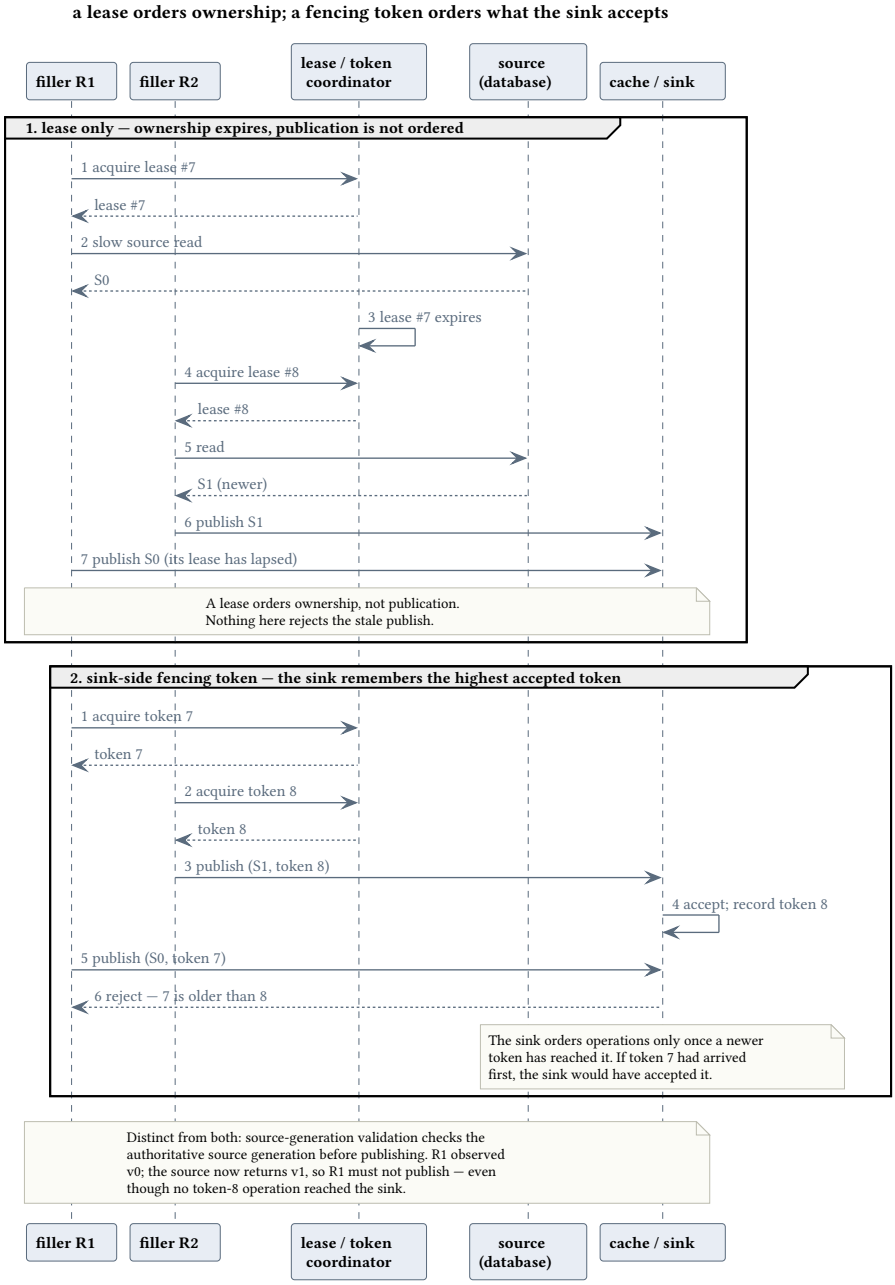


Figure 14: **A lease orders ownership; a sink-side token and a source-generation check refuse the stale publish.**

Form: sequence · Evidence: conceptual illustration

Text equivalent. Two panels plus a distinction note. **Lease only**: filler R1 holds lease 7, its source read outlasts the lease, R2 takes lease 8 and publishes S1, and R1 then publishes S0 — nothing in the lease rejects it, because a lease orders ownership rather than publication. **Sink-side fencing token**: R1 holds token 7 and R2 token 8; when R2's `(S1, token 8)` reaches the sink first, the sink records 8 and rejects R1's later `(S0, token 7)` — so the ordering depends on a newer token having actually reached the sink. **Distinct from both**, as the closing note says: source-generation validation checks the authoritative current generation before publishing and can refuse token 7 even when no token-8 operation has reached the sink. The figure is why the fill-lease card claims load coordination and not freshness, and it asserts no rate of its own. Source: `book/assets/sources/fig_lease_vs_fence.puml`

Publication fence — source-generation validation

Race closed	a reader that snapshotted committed state S0 republishes it **after** a writer commits S1 and invalidates
Mechanism	before publishing, verify that the source generation still equals the generation the reader observed; refuse the fill if it moved
Guarantee	the stale-fill race specifically — "publish after commit" is necessary and **not** sufficient. In the measured strict writer protocol the writer fences before and after the commit and treats the acknowledgement, not

	the commit, as the contract boundary; a strict contract can also be established without writer-side fencing, by synchronous source validation or an authoritative read
Does not solve	an unobserved writer, a dirty source read, or an indefinitely stale hit that is never revalidated

Evidence: active registered claim — `v2-15`

Source version token validated at the hit boundary

Race closed	a hit that never consults the source version
Mechanism	every hit compares the entry's version against the source's current version and misses on a mismatch
Guarantee	a process-local cache can be answerable: the hit carries its own proof — **provided** every relevant mutation advances the token atomically, including deletes, recreates and writers outside the application, and the validating read is authoritative enough for the declared contract
Does not solve	the read cost it adds, and a shared relaxed cache that never consults the token — an unconsulted token is decoration, not a guarantee

Evidence: mechanism demonstrated — the owned process-local arms of `20260921T-survey3`; no registered rate for the validation cost

Fill lease

Race closed	duplicate concurrent fills of the same key — a stampede
Mechanism	a bounded lease lets one filler work while the others wait, fall back or time out
Guarantee	Effect: duplicate fills are suppressed **while a valid lease is held**. Duplicate work can reappear if the lease expires or ownership is lost before the first filler finishes — which is exactly why a lease is not a fence
Does not solve	**any** freshness question. It is a load control; it guarantees nothing about the value filled

Evidence: newer evidence, signed but not yet registered — the churn configuration measured on 2026-09-23 demonstrates the lease path (`20260923T1100Z-churn1800-through`); no registry claim states a lease rate

TTL and early expiry

Race closed	an entry that would otherwise live forever, and the thundering herd that follows a mass expiry
Mechanism	a hard TTL bounds how long an entry is **eligible to serve**; probabilistic early expiry spreads the refill work before many entries reach the same deadline
Guarantee	Effect: logical validity is bounded, and refill work is spread. Neither is a hard upper bound on refill traffic
Does not solve	currentness. An entry inside its TTL is **permitted** to be stale, which is the opposite of a freshness proof, and early expiry does not by itself cap refill load

Evidence: active registered claim — `v3-01`, `v3-02`; hard expiry has still never been observed to fire

Transactional outbox

Race closed	a change lost between the commit and its asynchronous notification
Mechanism	the change is written in the same transaction as the mutation and consumed by the publisher, so the notification is durable and ordered after the commit
Guarantee	a committed change cannot disappear between the commit and its notification, and the delivery path is recoverable
Does not solve	freshness, and it is not universally required. **When** coherence depends on asynchronous publication, an outbox is what makes the committed change durably observable; strictness can also be established by synchronous source validation or an authoritative read, with no asynchronous step to protect. Even with an outbox, strict after-acknowledgement hits still need an acknowledgement policy and a read protocol that closes the window

Evidence: proposed / unmeasured — no committed measurement exists

15.4 What each mechanism is actually for

The dimensions are the point, and the header is short so no word breaks in a narrow column: **mutation** is whether the mechanism makes the source transition correct, **observation** whether a committed change is durably seen, **freshness** whether a read is proved current, **load** whether it changes how much work the system does, and **recovery** whether it helps reconcile after a failure. Reading a table like this is faster than holding eight cards in your head.

Mechanism	Mutation	Observation	Freshness	Load	Recovery
Pessimistic lock + recheck	yes	—	—	may cost	—
Conditional update / CAS	yes	—	—	may cost	—
Invalidation	—	partial: clears, does not fence	partial	—	—
Publication fence — source-gen. validation	—	—	yes, race-specific	—	—
Source version at hit boundary	—	—	yes	read cost	—
Fill lease	—	—	—	yes, while held	—
TTL / early expiry	—	—	bounds eligibility, not proof	yes	fallback only
Transactional outbox	—	yes	prerequisite in async designs	—	yes

Evidence status. *Active registered claim*: the active `book/evidence/v6/claims.json` states it. *Newer signed but unregistered evidence*: a signed 2026-09-23 study analysis demonstrates the mechanism, but no registry claim states a rate for it. *Mechanism demonstrated*: the harness or study demonstrates the mechanism without a registered number. *Proposed / unmeasured*: no committed measurement exists.

15.5 Freshness is a contract, not a speed knob

Every controlled strict/relaxed pair differed by −14.2% to +12.3% in read throughput — inside the run's single-trial noise. Mechanically a strict protocol adds write-path fencing, but the claim beside this paragraph measures read throughput only and does not quantify that cost, so quote no write-path figure from it. What the read result does forbid is the other direction: never restate this as "strict freshness is free".

v2-16-strict-freshness-read-cost · single_run_directional · read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop · full card in the evidence registry index

15.6 The gain, and what the gain leaves out

Adding a cache to warm cacheable reads bought 2.2–3.5x throughput and an order of magnitude in p99 (15.8–27.1 ms to 0.8–2.0 ms). One thing is left out of that number: the cache's own CPU and memory are not counted against the database side, so the ratio compares a database with a database **plus** a cache, rather than the same total resources arranged two ways (`conf-04`). A second comparison now counts the resources the other way — both sides given the same total — and there the same design reads 11k ops/s against 8.0k when the cache's resources are not counted. The two ways of counting are kept apart and never averaged together.

v2-14-cache-throughput-gain · single_run_directional · one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); this 2.2–3.5x range is supported only by the PostgreSQL single-node + Redis cells; do not transfer the range to YugabyteDB, to a cluster, or to open-loop demand · full card in the evidence registry index

v3-03-equal-total-framing-labelled · single_run_directional · one run per framing, one host, scale small; the two arms are single trials; the value of the arm is the label and separation, not a throughput ratio; the difference is inside the run's spread and is not attributed · full card in the evidence registry index

15.7 The failure mode, measured against the strict rule

Three logical application instances sharing one Redis cache on write-through + **relaxed** freshness recorded 87.81% wrong reads on the owned model and 87.24% on the legacy model, while every relaxed configuration running on a single instance recorded none.

What the counter counts. The harness field is named `wrong_reads`, and it counts reads that broke the strict rule stated above: a read that starts after a write is acknowledged must not return an older value. In this configuration the offending values had been committed, just earlier — which is exactly what choosing a

relaxed contract permits — and were **not** dirty, torn, uncommitted or impossible values. Read it as those broken promises, never as corruption: no configuration measured here produced an impossible value. Where a contract promises staleness bounded to a window, a further question arises that this run cannot answer — whether the stale values also stayed inside that window — so the counter is only evidence about staleness once the window is known.

The designs that keep the cache inside one process are the trap. The legacy process-local design's zero wrong reads are a **bypass** result, not safety: it served **0** cache hits across 15,578–21,756 reads, so there was effectively no cache to be wrong (`reports/20260921T-survey3.md`, the anchor of the registered claim). The owned process-local design did serve hits and checked the source version at the moment of the hit; those configurations recorded no broken promises. Checking the version at the hit is one way to obtain that guarantee — but whether it is the only way is not tested here, because no owned process-local design without the check was run. Read it as the mechanism these results are consistent with, not as a controlled removal of one part. A private in-memory lease cannot coordinate separate instances, and a source version token that is never consulted at the hit or at publication does not make a shared relaxed cache strict.

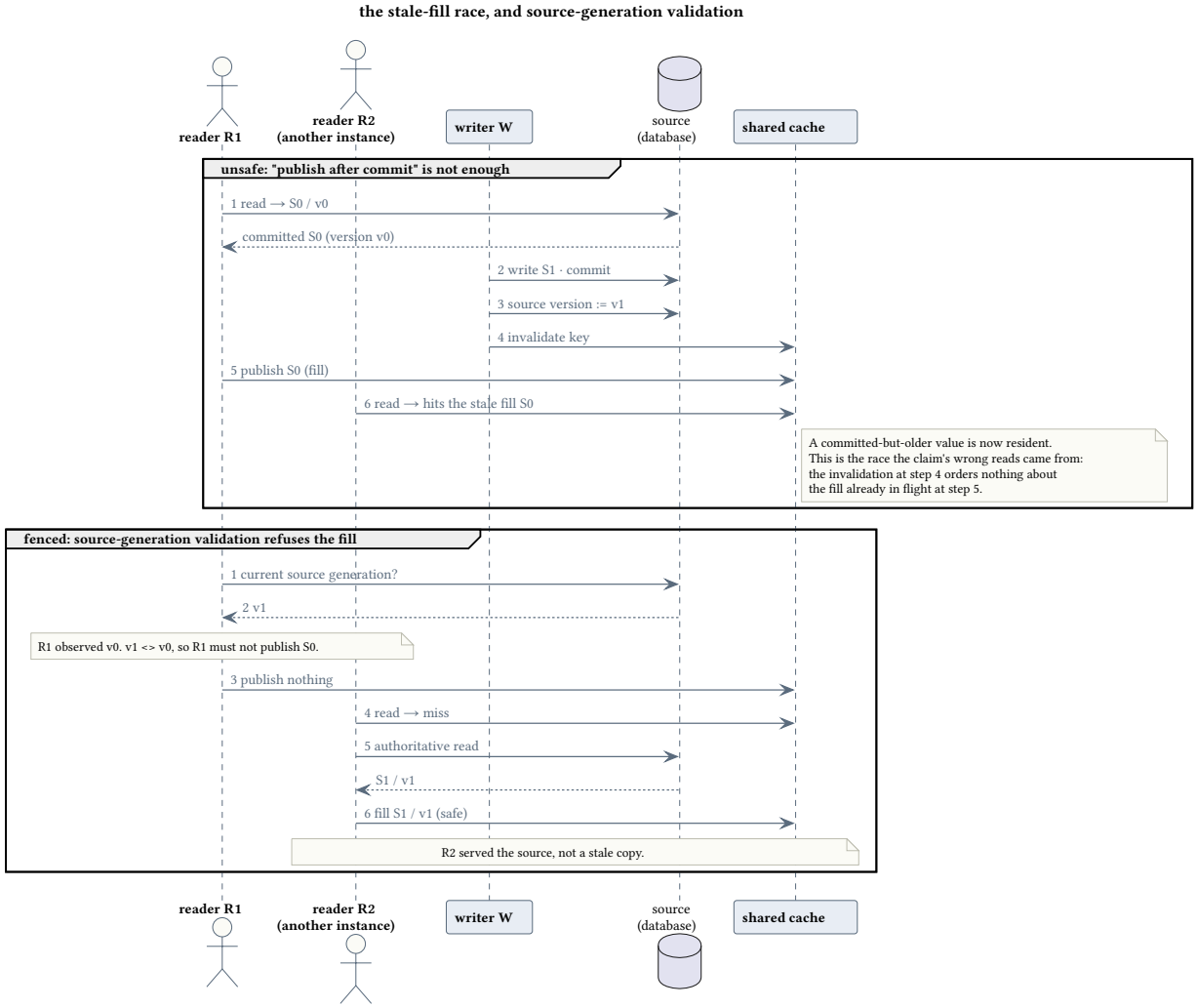


Figure 15: **The stale-fill race, and the source-generation check that closes it.**

Form: sequence · Evidence: observed result · illustrates v2-15

Text equivalent. Reader R1 snapshots committed state `S0 / v0`; writer W commits `S1`, advances the source version to `v1` and invalidates the key; R1, still holding `S0 / v0`, would then publish S0 as a fill. The fenced protocol validates the source generation before publishing: the source returns `v1`, `v1 != v0`, and R1 must not publish. A second reader R2 then misses the cache and takes an authoritative read of `S1 / v1`, which may safely be filled. The figure demonstrates the **stale-fill race** only; the strict-after-acknowledgement contract and its acknowledgement boundary are drawn separately in the freshness-timeline figure above. It illustrates the registered claim beside it and embeds no rate of its own. Source: `book/assets/sources/fig_cache_stale_fill.puml`

v2-15-three-instance-staleness · mechanism_supported · single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution · full card in the evidence registry index

15.8 Churn, engines and placement (2026-09-23)

Sustained churn crossed real 300 s TTL boundaries under load and stayed correct: one write-through configuration ran 1,801 s and crossed 6.0 boundaries with 0 wrong reads in 2,879,077 reads, and the two 601 s write-through / write-aside configurations crossed 2.0 each with 0 wrong reads. The hard-expiry path fired **0** times — only probabilistic early expiry did — so "an entry reached its hard TTL" is still not demonstrated and is carried as a gap below. The strict/relaxed pair now also runs per engine, with the client spread over the endpoints: on YugabyteDB 1-node strict read 514 against relaxed 596 app-99 ops/s, on 3-node 942 against 846; the direction differs per engine and neither is attributed. The colocated/non-colocated key pair favours the colocated layout (warm 882 against 673 ops/s, cacheable-99 852 against 520) as engine locality on one host, not verified colocation.

v3-01-churn-crosses-real-ttl · single_run_directional · one PostgreSQL + Redis cell at scale small; one sustained measurement, no repeat; hard expiry fired zero times; that clause is an open gap, not a confirmation; closed loop, so read volumes are a floor; the 5x-for-120 s burst has no write-rate knob and was not run · full card in the evidence registry index

v3-02-churn-600-through-vs-aside

external read copies / caches · single_run_directional

At 601 s the write-through and write-aside churn cells both crossed 2.0 real TTL boundaries with 0 wrong reads (1,467,721 reads / 197,196 writes for write-through; 1,364,997 reads / 182,282 writes for write-aside), and neither strategy is claimed to win: the study has no head-to-head fill/publication ratio.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: two separate single runs, not a controlled head-to-head pair; write-aside numbers come from a sibling run with its own digest; both are single trials; db-only framing; the cache container is outside the resource comparison here; hard expiry fired zero times in both cells

Source: studies/05-cache-consistency/reports/20260923T1020Z-churn600-through.md · run 20260923T1020Z-churn600-through · tag run/05-cache-consistency/20260923T1020Z-churn600-through

v3-04-yb-single-strict-relaxed

external read copies / caches · single_run_directional

On YugabyteDB 1-node (RF=1) one run recorded the relaxed write-through app-99 read at 596 ops/s against the strict cell's 514; the spread is large, this is a single run, and no freshness winner is attributed.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run per freshness policy on one engine; the strict/relaxed direction is not repeated; single endpoint (RF=1); this is not a cluster result; app-99 phase is a small mixed sample (spread 29.5% relaxed, 15.9% strict)

Source: studies/05-cache-consistency/reports/20260923T1152Z-res2-yb1.md · run 20260923T1152Z-res2-yb1 · tag run/05-cache-consistency/20260923T1152Z-res2-yb1

v3-05-yb-cluster3-strict-relaxed

external read copies / caches · single_run_directional

On YugabyteDB 3-node (RF=3), where the client spreads operations over all three endpoints, one run recorded the strict write-through app-99 read at 942 ops/s against the relaxed cell's 846 - the opposite direction from the 1-node pair - with each cell a single run and no engine ranked.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run per freshness policy; direction is not repeated and differs from the 1-node pair; three containers on one host, no real network; endpoint spread is round-robin by construction; app-99 phases have large spread (50.4% relaxed, 49.4% strict)

Source: studies/05-cache-consistency/reports/20260923T1157Z-res2-yb3.md · run 20260923T1157Z-res2-yb3 · tag run/05-cache-consistency/20260923T1157Z-res2-yb3

v3-06-colocated-vs-noncolocated-locality

rolldown / duplicated data · single_run_directional

On YugabyteDB 3-node (RF=3) a key-locality pair where the colocated layout (y1-coloc) keeps one donor's donations in one tablet and the non-colocated layout (y2-noncol) hashes them across tablets recorded the colocated arm faster on the warm read (882 vs 673 ops/s) and on the cacheable-99 read (852 vs 520), with 0 wrong reads either way. Physical tablet/leader placement was not evidenced, so this is engine locality evidence on one host, not verified colocation.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: the logical partition mapping follows deterministically from the colocated table's PRIMARY KEY ((person_id) HASH, ...), so one donor's rows hash to one tablet by configuration, not by observed placement; physical tablet/leader placement was not observed: the runner could not produce a tablet/leader distribution (its placeholder held only the three server names), so this is engine locality on one host, not verified colocation; all containers run on one WSL2 host; one host is not colocation evidence; one run per layout, no cache in this scenario, no repeat; spreads are large (warm 40.0% y1, 22.2% y2)

Source: studies/05-cache-consistency/reports/20260923T1215Z-res2-placement-yb3.md · run 20260923T1215Z-res2-placement-yb3 · tag run/05-cache-consistency/20260923T1215Z-res2-placement-yb3

15.9 What is still open

Coverage gap

The hard TTL has still never been observed to remove an entry; medium scale beyond the harness maximum, the tablet/leader placement distribution and open-loop arrival demand are unmeasured. Treat these as open, never as null results.

v3-gap-01-study05-remaining-regimes · gap · these are coverage gaps, not null results; medium scale is an order of magnitude beyond the current generator; placement verification needs an instrument the runner does not have; open-loop demand is owned by task-20260922T200831Z-study05-v2-open-loop · full card in the evidence registry index

15.10 Boundaries

Redis was measured as a cache, never as an authoritative key-value store. JSONB is not a native document store. Transfer either result to a native family only as a labelled analogy.

15.11 The checklist

The eight questions, answered for this concept; the status in brackets is the evidence behind the answer.

- **Invariant.** Every cache value corresponds to a committed database state, and under the strict contract a read that starts after an acknowledged write never returns an older committed value. [direct]
- **Where the information lives.** In the authoritative row, and in a copy in Redis or in process memory. The copy is never the source of truth. [direct]
- **Where the contention is.** One hot key and its fill lease: who is allowed to fill, and who must wait. [direct]
- **What work moved from reads to writes.** Invalidation and publication. A write becomes responsible for the copies of the value it changed. [mechanism]
- **What state is derived.** The cached payload and its version or source generation. [mechanism]
- **What can become stale.** Any value a relaxed contract is permitted to return — with three instances sharing a relaxed write-through cache this was 87% wrong reads, against zero for one instance. [direct]
- **Where it stops working.** When an invalidation cannot reach every instance, and at the boundary the corpus has not crossed: the hard TTL has never been observed to remove an entry. [gap]
- **The experiment that would settle it.** Strict/relaxed pairs under the same backend, crossed with instance count, beside the stale-invalidation control. [direct]

16 Distributed placement

CONCEPT · DISTRIBUTED PLACEMENT

Node count, replication factor and actual data placement are three separate facts. A 3-node cluster on one laptop has the first two and none of the third's network consequences.

16.1 What the corpus measures

Studies 01–03 run PostgreSQL single-node and YugabyteDB single-node and 3-node RF=3 cells. The multi-node cells show placement-shaped effects — a section-sharded layout collapsed point lookups to 26 ops/s on YugabyteDB — but the balance of tablets across nodes is unexplained.

Study 05 adds two things those cells did not have: a key-locality pair, where the same donor's donations are held in one tablet in one layout and hashed across tablets in the other, and three-node cells whose client spreads operations over all three endpoints rather than one.

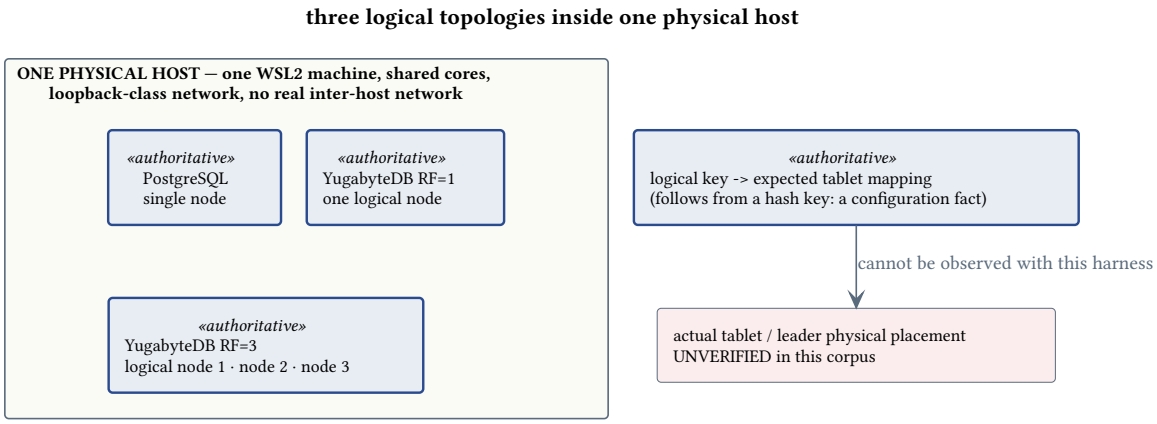


Figure 16: **Logical multi-node topology; no real inter-host network was measured.**

Form: structure · Evidence: conceptual illustration

> **Text equivalent.** PostgreSQL single-node, YugabyteDB RF=1 and YugabyteDB RF=3 are drawn as three logical topologies, but every Yugabyte container sits inside one physical host boundary and talks over a loopback-class path, so the picture is **not** three independent machines. The lower band separates a logical mapping — a hash key places one person's rows in one tablet **by configuration** — from physical tablet and leader placement, which is marked UNVERIFIED in this corpus. The figure asserts no measured value.

16.2 What it does not measure

Two different limitations, and they should not be collapsed into one sentence.

Physical placement is unverified. The locality pair ran, but the runner produced no tablet or leader distribution, so the layout labels record intent, not observed placement. That is an **instrument** gap: it needs a measurement the harness cannot take, not just another run.

Endpoint distribution is partial. The 2026-09-23 Study 05 three-node cells spread operations round-robin over three endpoints, but the earlier multi-node cells still reach the cluster through a single endpoint. Their ratios remain valid for a client that does one endpoint's worth of cluster access, and none of it is a network result.

Both are recorded gaps, not null results.

v4-gap-01-physical-placement-unverified

rolldown / duplicated data · gap · instrument gap + coverage gap

Physical tablet and leader placement remains unverified even though the Study 05 key-locality pair has now been executed: v3–06 records the colocated layout ahead on both reads, but its runner could not produce a tablet/leader distribution, so the

layout labels are intent rather than observed placement. Study 04's y1/y2 pair is still unexecuted, Study 01 D7/D24 and Study 03 L3 remain placement intent with measured effects, and every node still runs as a container on one host.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: containers on one laptop are not evidence of colocation; the runner could not produce a tablet/leader distribution, so the placement labels are intent, not observed placement; the locality pair is one run per layout with large spreads

Source: studies/05-cache-consistency/reports/20260923T1215Z-res2-placement-yb3.md · run 20260923T1215Z-res2-placement-yb3 · tag run/05-cache-consistency/20260923T1215Z-res2-placement-yb3

v4-gap-02-endpoint-distribution-partial

normalized facts · gap · coverage gap + instrument gap

Client endpoint distribution is measured only for the 2026–09–23 Study 05 three-node cells, where the client spreads operations round-robin over all three endpoints (v3–05). The earlier multi-node cells still reach the cluster through a single endpoint: the study 01 and study 02 yb-cluster3 runs each set one DSN against yb-n1. Their cluster ratios therefore still stand in for cluster access through one endpoint, the Study 05 spread is recorded as a node count rather than a per-endpoint balance, and no run has a real network between nodes.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: no real network between nodes; endpoint distribution is unmeasured for the studies 01–03 multi-node cells, which connect through a single endpoint; the Study 05 three-node spread is round-robin by construction and is recorded as a node count, not as a measured per-endpoint balance

Source: studies/05-cache-consistency/reports/20260923T1157Z-res2-yb3.md · run 20260923T1157Z-res2-yb3 · tag run/05-cache-consistency/20260923T1157Z-res2-yb3

v2-gap-06-native-datastore-families · gap · do not transfer the embedded-document or cache results to a native family · full card in the evidence registry index

16.3 Boundaries

Do not read a single-node-versus-cluster ratio as production capacity guidance. The nodes share the host's cores and have no real network between them.

16.4 The checklist

Every answer below is a gap, and the honest response is to say so rather than to reason from how clusters usually behave.

- **Invariant.** The same logical read returns the same answer wherever the data sits. [gap]
- **Where the information lives.** In engine-chosen tablets and leaders — and the corpus records layout **intent**, never an observed distribution. [gap]
- **Where the contention is.** Unknown: no tablet or leader distribution has been measured. [gap]
- **What work moved from reads to writes.** Replication and placement move work and latency, but no experiment in the corpus separates them from node count. [gap]
- **What state is derived.** A lagging replica is a copy rather than derived state, and no control has been seen to serve it. [gap]
- **What can become stale.** A follower, in principle. Unmeasured, therefore unstated. [gap]
- **Where it stops working.** Unknown on one host with no real network path: a container count is not a topology. [gap]
- **The experiment that would settle it.** Independent hosts, balanced endpoints, a recorded tablet/leader distribution and injected latency — Study 07's axes. [gap]

17 Reading benchmarks honestly

CONCEPT · READING BENCHMARKS

A benchmark answers the question its harness asks. Most wrong conclusions in this corpus came from reading a number as if it answered a different question, not from a bad run.

17.1 Six questions to ask of any number

- **Closed or open loop?** Every race here is closed loop. Closed-loop load can understate fixed-rate behaviour, so its tail must not be read as an SLO prediction.
- **One trial or many?** The scatter between repeats inside one measurement is not the same as the scatter between separate runs of the whole experiment.
- **Were both sides given the same resources?** Comparing one node with three answers "what if I add machines", not "what if I spend the same total".
- **In what order were the designs run?** In Study 02 the two designs swap places on PostgreSQL depending on the order, so no ratio is quotable until an order-controlled repeat is run.
- **Did the check that is meant to fail actually fail?** A correctness check that no deliberately broken design ever trips proves less than it appears to.
- **One configuration, or the whole run?** A run's totals and failures belong to the run; a claim cites a selected subset of what was measured.

17.2 Worked examples of a number that cannot be quoted

v2-09-postgres-ledger-ratio-unsettled · single_run_directional · buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted · full card in the evidence registry index

v2-gap-01-ledger-load-multiplier-unstable · gap · a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted · full card in the evidence registry index

v2-16-strict-freshness-read-cost · single_run_directional · read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop · full card in the evidence registry index

17.3 Before quoting a number

A table, not a figure: check it against the card beside the number.

Question	Where the answer lives	If it is missing
Which claim does this number belong to?	The claim id in the card	Do not quote it.
Was it one run, or several?	`trials` on the card	Say "one run, one trial" and stop.
Closed or open loop?	The report's workload section	Do not read the tail as an SLO.
Same resources on both sides?	`resource_framing` in the result	Never mix the two ways of counting.
One decision, or several changes at once?	The card's own label	Attribute nothing to a single change.
Did the deliberately broken control fail?	The run's verification block	Treat correctness as unproven.
What else changed at the same time?	`confounds` on the card	Do not attribute the effect.
What gap does it leave?	The gap's `gap_kind`	Name the kind, never "unknown".

17.4 Boundaries

Laptop topologies with shared cores are a controlled comparison, not a capacity model. The environment caveat belongs wherever a conclusion is drawn.

17.5 The checklist

The eight questions applied to a **number** rather than to a design; the status in brackets is the evidence behind the answer.

- **Invariant.** A number is reproducible from its commit, its pinned images and its inputs digest, or it is not evidence. [mechanism]
- **Where the information lives.** In the run manifest, the result JSON, the plans and the report — not in the prose that quotes it. [mechanism]
- **Where the contention is.** The machine itself: one measurement at a time, under the benchmark lock. [mechanism]
- **What work moved from reads to writes.** Nothing; this concept prices work, it does not move it. [mechanism]
- **What state is derived.** A blended score is derived from its parts and must be decomposed before it is quoted. [mechanism]
- **What can become stale.** A report whose digest no longer matches the results it describes, and any number copied out of a superseded analysis. [mechanism]
- **Where it stops working.** Single-trial cells, closed-loop tails read as service levels, differences inside the noise floor, and cells whose own trials disagree by more than the 20% rule. [mechanism]
- **The experiment that would settle it.** Independent runs with randomized order and reported across-run spread — which is what the first replication in the corpus did. [direct]

18 Glossary

APPENDIX · GLOSSARY

The single definition site for the vocabulary the rest of the book uses. Where a term is also a registry field, the registry's value is named in brackets.

Term	Meaning
claim	One reviewed statement about what the corpus measured, registered in the active evidence package and identified by an id such as `v2-16`. A number may not appear in this book without one.
strength	How strongly a claim is supported: `single_run_directional` (one run, one trial per cell, a direction not a curve), `repeated_controlled` (repeated fresh loads with a control), `mechanism_supported` (the mechanism is shown, the rate is not), `gap` (an explicit absence of evidence).
direct evidence	A callout whose numbers come from the cited run of the claim beside it.
mechanism evidence	A card or callout that demonstrates how something works without establishing a rate.
analogy	A labelled transfer beyond the measured family or environment. Never a measurement.
gap	An explicit absence of evidence, of one of four kinds. A gap is never a null result.
schema limitation	The representation cannot answer the question, so no run would. The hold-funnel gap is one.
coverage gap	A relevant regime has not been measured yet, but could be.
instrument gap	The property could not be observed with the tools the study had. Physical tablet placement is one.
unstable measurement	The number moved between runs with no code change, so no value is quotable.
partially superseded	Part of the claim has been replaced by newer evidence; the card says what moved to which successor and what is still open.
retired	The claim is no longer active, is not cited, and names the successor that closes it.
run	One execution of the harness under the benchmark lock, identified by a run tag and an inputs digest. Results are comparable only within one environment.
replication	A whole-run repeat of the same cell. The cards report it as `whole_run_replications`; `Trials: 1` means exactly one.
trial	A measurement inside a run. A fresh-load trial repeats the load phase; an in-run repeat is another measurement window inside one process. Neither is an independent replication.
read score	A blended within-run composite of a family's read queries. It compares variants inside one run and nothing else: it is not the throughput of any single query and is not quotable across runs.
wrong_reads	The harness counter for reads that violated the strict comparator in force, not for corruption. A relaxed cell can record many without serving an impossible value.
strict after acknowledgement	`begin(R1) > ack(W1)` implies the read must not return a version older than `W1`, unless a later write to the key supersedes it. This is the comparator the harness counts against.

Term	Meaning
relaxed freshness	A family of named weakenings of the strict line: session/read-your-writes, bounded staleness with a declared delta, eventual freshness, and best effort. Bounded staleness is one member, not the definition.
resource framing	`db-only` counts the database's resources; `equal-total` gives the cache a share of an equal budget. The two are never pooled, and a single-node versus cluster comparison answers "add machines", not "same budget".
colocation	A design intent that a key's rows share a partition. Logical mapping can follow from the schema; verified physical placement requires an instrument the studies did not have.
rollup	A parent aggregate maintained from its children. Where it lives in the same database it is updated in the same transaction; otherwise it is published, with a named delivery and reconciliation path.
rolldown	Parent information copied onto the child to avoid a join.
embedding	A child collection stored inside its parent row, as JSON or an array. In this corpus that means a relational JSON column, not a native document database.
source correctness	A mechanism that makes the authoritative mutation itself correct, such as a lock plus invariant recheck or a conditional update.
copy correctness	A mechanism that orders what leaves the transaction: invalidation, a publication fence, version validation at the hit boundary, a fill lease, a TTL, or an outbox.
derived state	A stored structure that answers from current facts and owes its maintainer a correctness contract on every contributing write: an index, a rollup, a rolldown or an external read copy.
retained history	Information kept so a question can be answered after the event was undone: an event/ledger table, an audit trail, a temporal history or a snapshot. A current-state-only representation cannot reconstruct what it has destroyed.
invalidation	Removing or marking a cache entry stale. It orders nothing about a read or a fill already in flight.
fill lease	A bounded ownership period that suppresses duplicate concurrent fills while it is valid and honoured. It is a load control, not a freshness proof.
fencing token	A monotonically increasing token checked at a protected sink, which rejects an operation whose token is older than one the sink has already accepted. It orders operations at the sink; it does not itself consult the source.
source-generation validation	Before publishing, checking that the authoritative current generation still equals the generation the reader observed, and refusing the fill if it moved. This closes the stale-fill race.
cache CAS	A conditional modification of cache state against an explicitly stated expected value or generation. It is not source-generation validation.
hit validation	Verifying a cached value against authoritative or version state at the hit boundary before serving it.
transactional outbox	Writing a change in the same transaction as the mutation so it can be published durably. It makes a committed change observable; it does not by itself make every cache hit current.
TTL / early expiry	A bound on how long an entry is eligible to serve, plus probabilistic early refill before many entries reach one deadline. It controls eligibility and load, not currentness, unless the declared contract is itself bounded by it.

Term	Meaning
publication fence	Refusing a fill whose source moved after the reader took its snapshot. This, not invalidation, is what closes the stale-fill race; source-generation validation is its strongest named form.
confound	A second change that moved with the one being measured. Registered in the package's confound register and published beside the numbers it affects.
family, variant, scenario	A family is a way of deciding where a fact lives; a variant is one design inside a family; a scenario is a workload built around one question, such as a hot drop or a refund.

19 Evidence registry (active v6 claims)

The active registry holds 31 claims that may be cited as evidence. This index is rendered from `book/evidence/v6/claims.json` at build time; the digest above identifies its content.

19.1 normalized facts

v2-01-normalized-index-first

normalized facts · single_run_directional

In the charity -> person -> donation tree, the first large read win is the query index: D2 indexed scores 13.3x D1 minimal's read score; flattening the foreign key instead (D3 flat+FK) reaches 20.5x.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-01-study01-d2d3-copied-key-package

Limits: PostgreSQL 1 node and YugabyteDB 1/3 nodes in one run; small scale; one trial per cell; read score is a blended measure, not one query

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-06-hot-drop-designs

normalized facts · single_run_directional

For a hot drop, pre-created seats taken by compare-and-set or SKIP LOCKED (P3 CAS, P2 skip-locked) sell 4–8x faster than any single counter row, but only with retry backoff in mind, and a counter on the event row also blocks unrelated edits to it. Both deliberately wrong controls fired in the same run.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run, one trial per cell; the run has 42 cells of which 2 failed (C2 count (SER) on the two YugabyteDB topologies); those cells are not evidence; hot-drop workload only

Source: studies/02-ticket-booking/reports/20260913T021206Z.md · run 20260913T021206Z · tag run/02-ticket-booking/20260913T021206Z

v2-10-guarded-confirm-refusals

normalized facts · single_run_directional

On YugabyteDB a guarded confirmation is not enough: correct designs refused valid holds, and retrying the refused statement once removed every refusal; on PostgreSQL the guarantee held everywhere. This claim is about refusals, not throughput, so no cost conclusion follows from it.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: v1 matrix run, one trial per cell; the claim is about refusals, not throughput; retry-once is a design variant (S1r), not a tuning flag

Source: studies/03-reserved-seating/reports/20260915T002411Z.md · run 20260915T002411Z · tag run/03-reserved-seating/20260915T002411Z

v2-12-lock-vs-version-upper-bound

normalized facts · single_run_directional

Under one hot key with 16 writers a lock-before-update (c2_pessimistic_lock) delivered 1.76x the throughput of a version check (c1_optimistic_version), an UPPER BOUND on the lock's advantage because the run could not separate it from extra per-retry harness work; an unchecked read-modify-write (x1_lost_update_control) lost 93.7% of acknowledged updates.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one key, one installed product, 16 writers; PostgreSQL single node, small scale; one trial; -retries 1 control not run; 1.76x is one point on a writer curve, not a curve

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v2-gap-03-study04-missing-designs

normalized facts · gap · coverage gap

Eight of the eighteen planned Study 04 configuration-portal designs are not implemented, and cardinality, cadence/churn, repeated randomized trials, equal-total resources, placement and YugabyteDB are unmeasured. These are coverage gaps, not null results.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: no conclusion may read 'the missing designs would not change this'; owned by task-20260922T025912Z-study04-v2-protocol

Source: studies/04-configuration-portal/reports/20260921T1215Z-small.md · run 20260921T1215Z-small · tag run/04-configuration-portal/20260921T1215Z-small

v4-gap-02-endpoint-distribution-partial

normalized facts · gap · coverage gap + instrument gap

Client endpoint distribution is measured only for the 2026–09–23 Study 05 three-node cells, where the client spreads operations round-robin over all three endpoints (v3–05). The earlier multi-node cells still reach the cluster through a single endpoint: the study 01 and study 02 yb-cluster3 runs each set one DSN against yb-n1. Their cluster ratios therefore still stand in for cluster access through one endpoint, the Study 05 spread is recorded as a node count rather than a per-endpoint balance, and no run has a real network between nodes.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: no real network between nodes; endpoint distribution is unmeasured for the studies 01–03 multi-node cells, which connect through a single endpoint; the Study 05 three-node spread is round-robin by construction and is recorded as a node count, not as a measured per-endpoint balance

Source: studies/05-cache-consistency/reports/20260923T1157Z-res2-yb3.md · run 20260923T1157Z-res2-yb3 · tag run/05-cache-consistency/20260923T1157Z-res2-yb3

v2-17-optimistic-pessimistic-crossover

normalized facts · single_run_directional

Under one hot key, a bounded-retry version check (c1_optimistic_version) and a lock-before-update (c2_pessimistic_lock) cross over between one and two concurrent writers: the lock delivers 0.89x the version check's throughput at 1 writer (1,177 against 1,329 ops/s) and 1.10x at 2 writers (1,285 against 1,167), rising to 1.68x at 32 writers (877 against 523) and 1.60x at 64, while the version check's own throughput falls 2.9x across the sweep (1,329 to 466 ops/s) and its conflict count rises from 0 to 21,301.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale, one hot key, one measurement per level across seven runs (1, 2, 4, 8, 16, 32 and 64 writers), so the curve carries no spread at any level; the retry budget was tested as the mechanism and rejected: with -retries 0 the c2/c1 ratio at 16 writers is 1.60 against 1.61 with retries, and c2_pessimistic_lock - which never retries - moved by the same 10% as c1 between the two same-day arms, so the lock's advantage is not attributable to retry work (analysis 20260926T-retry-control, runs 20260926T142926Z and 20260926T143014Z); the seven runs used two phase sets (the first four also ran read and write phases); the contention metric should not depend on that, but it is an assumption rather than a measurement; no negative control in this sweep; control_coverage is recorded absent rather than implied; the 1-writer point is the edge of the sweep, so the optimistic side is bracketed by one level only; absolute numbers are a laptop comparison; the shape of the curve is what transfers; absolute throughput is day-specific: c2 measured 869 ops/s on 2026–09–23 and 1,056 on 2026–09–26 at identical settings, phases and writer count, so only the within-run ratio is carried across days

Source: studies/04-configuration-portal/reports/20260923T0011Z-ctlC-w1.md · run 20260923T0011Z-ctlC-w1 · tag run/04-configuration-portal/20260923T0011Z-ctlC-w1

19.2 rolldown / duplicated data

v2-05-d3-read-advantage-repeated

rolldown / duplicated data · repeated_controlled

D3's (d3_flattened_fk) read advantage over the normalized indexed design survives five fresh loads; the recency SQL and actual heap avoidance explain the gain, while its insert cost remains imprecise.

Trials: 1 whole-run replication · 5 fresh-load trials per cell · 1 inner iteration

Limits: five fresh-load trials per cell, one session; the insert-cost estimate is imprecise and must not be quoted as a ratio

Source: studies/01-charity-tree/reports/20260913T125342Z-v3.md · run 20260913T125342Z-v3 · tag run/01-charity-tree/20260913T125342Z-v3

v3-06-colocated-vs-noncolocated-locality

rolldown / duplicated data · single_run_directional

On YugabyteDB 3-node (RF=3) a key-locality pair where the colocated layout (y1-coloc) keeps one donor's donations in one tablet and the non-colocated layout (y2-noncol) hashes them across tablets recorded the colocated arm faster on the warm read (882 vs 673 ops/s) and on the cacheable-99 read (852 vs 520), with 0 wrong reads either way. Physical tablet/leader placement was not evidenced, so this is engine locality evidence on one host, not verified colocation.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: the logical partition mapping follows deterministically from the colocated table's PRIMARY KEY ((person_id) HASH, ...), so one donor's rows hash to one tablet by configuration, not by observed placement; physical tablet/leader placement was not observed: the runner could not produce a tablet/leader distribution (its placeholder held only the three server names), so this is engine locality on one host, not verified colocation; all containers run on one WSL2 host; one host is not colocation evidence; one run per layout, no cache in this scenario, no repeat; spreads are large (warm 40.0% y1, 22.2% y2)

Source: studies/05-cache-consistency/reports/20260923T1215Z-res2-placement-yb3.md · run 20260923T1215Z-res2-placement-yb3 · tag run/05-cache-consistency/20260923T1215Z-res2-placement-yb3

v4-gap-01-physical-placement-unverified

rolldown / duplicated data · gap · instrument gap + coverage gap

Physical tablet and leader placement remains unverified even though the Study 05 key-locality pair has now been executed: v3–06 records the colocated layout ahead on both reads, but its runner could not produce a tablet/leader distribution, so the layout labels are intent rather than observed placement. Study 04's y1/y2 pair is still unexecuted, Study 01 D7/D24 and Study 03 L3 remain placement intent with measured effects, and every node still runs as a container on one host.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: containers on one laptop are not evidence of colocation; the runner could not produce a tablet/leader distribution, so the placement labels are intent, not observed placement; the locality pair is one run per layout with large spreads

Source: studies/05-cache-consistency/reports/20260923T1215Z-res2-placement-yb3.md · run 20260923T1215Z-res2-placement-yb3 · tag run/05-cache-consistency/20260923T1215Z-res2-placement-yb3

19.3 rollups / materialized aggregates

v2-02-rollup-family

rollups / materialized aggregates · single_run_directional

Parent rollups win the aggregate read score (D4 rollup/trg 73.8x and D5 rollup/app 69.9x against D1 minimal) and pay for it on inserts (2.6k/s and 2.1k/s against 6.2k/s).

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run, one trial per cell; PostgreSQL 1 node numbers shown; YugabyteDB in the same report; the write cost is per-insert on a fresh load

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-04-recency-maintained-index

rollups / materialized aggregates · single_run_directional

The recency flag works, but an index on the rollup column answers the same question faster: comparing d20_recency_flag with d22_recency_rollup_idx in the same run gives the reported read speedup on PostgreSQL and YugabyteDB.

Trials: 1 whole-run replication · 3 fresh-load trials per cell · 1 inner iteration

Confounds: conf-02-study01-d20d22-write-axis

Limits: the write comparison is confounded: d22 also maintains D4's multi-column rollup package, so its write cost is not the index alone; three trials per cell; claim-02's v1 trials field was recorded as 1 and is corrected here

Source: studies/01-charity-tree/reports/20260916T090036Z-v3.md · run 20260916T090036Z-v3 · tag run/01-charity-tree/20260916T090036Z-v3

v2-13-trigger-rollup-cost

rollups / materialized aggregates · run_replicated

Maintaining the portal overview with a per-row trigger (n3_rollup_trigger) costs 4.0–5.7x the application-maintained rollup (n4_rollup_app) on whole-configuration replacement across three randomized-order runs (median 4.94x: n4 518–635 ops/s against n3 91–159 ops/s), and the trigger's latency tail is 22–46x worse (p99 61–109 ms against 2.2–2.7 ms); the trigger cell's own trials disagree by 35–115%, so this is a range over an unstable cell, and the earlier single-run 6.85x figure did not reproduce.

Trials: 3 whole-run replications · 3 fresh-load trials per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale; three whole runs, three trials per cell in each; run-level replication only: separate runs on one machine, not hardware-independent, and still exposed to this host's measurement hazards; the trigger cell's own trials disagreed by 35–115% across the three runs, above the ~20% precision rule, so only the direction and an order-of-magnitude range are supported; the ratio is not a point estimate; the ~15x read gain belongs to the earlier single run and was not re-measured in the replication; the earlier single-run 6.85x figure did not reproduce and must not be quoted; the trigger's product is a structural guarantee, not speed; the three runs realized only two distinct design orders (seeds 202 and 303 coincided, and in the sibling read tranche all three seeds coincided); randomized order describes the method, and the realized order is recorded in each manifest; the penalty is size-dependent, not a constant: crossing the same pair with entries per installed product gives 2.07x at 10 entries, 4.38x at 60 and 24.20x at 240, so this ratio holds at the small scale default of 60 entries and must not be quoted as a property of the design without the size (v2–18-maintenance-size-interaction)

Source: studies/04-configuration-portal/reports/20260925T190831Z.md · run 20260925T190831Z · tag run/04-configuration-portal/20260925T190831Z

v2-18-maintenance-size-interaction

rollups / materialized aggregates · single_run_directional

Crossing who maintains the parent rollup (trigger n3_rollup_trigger against application n4_rollup_app) with the size of a replacement (10, 60 and 240 entries per installed product) shows the maintenance penalty is a function of size rather than a constant: the application design delivers 2.07x the trigger design on whole-configuration replacement at 10 entries (952 against 459 ops/s), 4.38x at 60 (631 against 144) and 24.20x at 240 (242 against 10), while the trigger's p99 rises from 3 to 156 ms against the application's 1 to 8 ms and no rollup mismatch is recorded at any size.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: PostgreSQL single node, small scale, one measurement per tier across 10, 60 and 240 entries per installed product, so the curve carries no spread at any level; one operation only: w05_replace_all replaces the whole configuration, and a workload that appends a single entry would sit at the low end of this curve at any size; the axis is capped by the small scale at 240 entries and the curve is still rising steeply there, so nothing here bounds where it flattens; no negative control in these three runs; control_coverage is recorded absent rather than implied; two designs cannot separate the two mechanisms that predict this shape: the trigger fires once per row written, and its per-row work may also be heavier than the application's single statement; the tier-60 cell reads 4.38x, inside the replicated range v2–13 measured for the same operation at the same tier on another day (3.99–5.69x); that cross-day comparison carries the approximately 30 percent day term the retry control measured

Source: studies/04-configuration-portal/reports/20260926T152621Z.md · run 20260926T152621Z · tag run/04-configuration-portal/20260926T152621Z

19.4 embedded documents

v2-03-embedding-not-ahead

embedded documents · single_run_directional

Embedding the donation child in the donor row (D6 embedded) never came out ahead of the normalized designs on reads: 2.4x D1 minimal's read score, below both indexed designs, with a 0.57x insert rate.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run, one trial; embedding here means a JSON/array child column, not a native document database

Source: studies/01-charity-tree/reports/20260912-small.md · run 20260912-small · tag none (legacy)

v2-11-section-seat-map

embedded documents · single_run_directional

On YugabyteDB the section document (L2 document) answers the 100k-seat map 4.5–11x faster than per-seat rows, while the section-sharded variant (L3 sharded) collapses point lookups to 26 ops/s; PostgreSQL shows no attributable difference. This is a family/package comparison, not a one-decision pair.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-03-study03-layout-family-package

Limits: one read-only run, one trial, 5 s per measurement; quota-bound nodes; read-only phases; no control could fire; differences below ~1.8x are not attributable

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

v2-gap-06-native-datastore-families

embedded documents · gap · coverage gap

No native document, key-value, graph or wide-column database was measured: JSONB/state is not evidence for a native document database, and Redis was measured only as a cache. Cross-family transfers to native datastores are analogy, not evidence.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: do not transfer the embedded-document or cache results to a native family

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

19.5 append-only history / snapshots

v2-07-refund-answerability-and-storage

append-only history / snapshots · single_run_directional

A ticketing database chosen only to prevent overbooking cannot report refunds at all: in 14 of 15 designs the refunds window is unanswerable and the sales/revenue and last-purchase reports are wrong across a refund. The append-only ledger (X1 CAS+ledger) that fixes it costs about 33–35% more storage.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: answerability is a schema fact, not a timing; the +33–35% storage delta is a relation-size difference measured in the primary run and deterministic for the same dataset and design; it is not a repeated timed result; the P3 -> X1 read rows of this run are not attributable (cell-level effect) and are excluded

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-08-ledger-race-cost-128-buyers

append-only history / snapshots · single_run_directional

At 128 buyers the ledger costs 1.1–2.3x of the sell-out race on PostgreSQL and 1.8–2.2x on YugabyteDB with tight spreads, 2.1–2.5x on the refund itself, and not one under-booked seat anywhere; it delays the organiser's own edit by up to 5x.

Trials: 1 whole-run replication · 3 fresh-load trials per cell · 128 inner iterations

Confounds: conf-05-study02-buyer-arms-disagree

Limits: one small run, closed loop, quota-bound nodes; scoped to 128 buyers; must not be extrapolated to 32 buyers; single run, three trials per tier

Source: studies/02-ticket-booking/reports/20260921T205212Z.md · run 20260921T205212Z · tag run/02-ticket-booking/20260921T205212Z

v2-09-postgres-ledger-ratio-unsettled

append-only history / snapshots · single_run_directional

The two arms of the P3/X1 pair disagree in sign on PostgreSQL (32 buyers with P3 first: X1 faster; 128 buyers with X1 first: P3 faster), and the 32-buyer arm's own P3 trials disagree by 92–283%. No PostgreSQL ledger-cost ratio is quotable from this study; the run's own YugabyteDB range is 1.7–5.7x (not the headline's narrower 1.7–2.1x).

Trials: 1 whole-run replication · 3 fresh-load trials per cell · 32 inner iterations

Confounds: conf-05-study02-buyer-arms-disagree

Limits: buyer count and design order changed together, so neither can be isolated; one arm is a partial cell on yb-cluster3 (two-trial medians); a repeated-design interleaved control is required before any PostgreSQL ratio is quoted

Source: studies/02-ticket-booking/reports/20260921T183722Z.md · run 20260921T183722Z · tag run/02-ticket-booking/20260921T183722Z

v2-gap-01-ledger-load-multiplier-unstable

append-only history / snapshots · gap · unstable measurement

The append-only ledger's load-time multiplier is not published: the same pair and loader measured X1's pg-single load at 7.8x P3's in one run and 1.2x in the two sibling runs, with an empty harness diff.

Trials: 3 whole-run replications · 1 fresh-load trial per cell · 1 inner iteration

Limits: a single cell's one-shot load is not a design cost; requires its own repeated load-only cell before any multiplier is quoted

Source: studies/02-ticket-booking/reports/20260920T234953Z.md · run 20260920T234953Z · tag run/02-ticket-booking/20260920T234953Z

v2-gap-02-hold-funnel-unanswerable

append-only history / snapshots · gap · schema limitation

No Study 03 design can answer the hold funnel report that would size the 40-minute hold window: releasing an expired hold clears the row, so an expired hold is indistinguishable from a seat never held.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: a schema limitation, not a missing measurement

Source: studies/03-reserved-seating/reports/20260921T231328Z.md · run 20260921T231328Z · tag run/03-reserved-seating/20260921T231328Z

19.6 external read copies / caches

v2-14-cache-throughput-gain

external read copies / caches · single_run_directional

Adding a cache to warm cacheable reads buys 2.2–3.5x in throughput and an order of magnitude in p99 (15.8–27.1 ms to 0.8–2.0 ms). This is an add-cache result under resource_framing db-only, not equal-total resource efficiency, and Redis here is a shared cache, never an authoritative key-value store.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-04-study05-db-only-cache

Limits: one trial per measurement, one machine, closed loop; small scale (800 donors), PostgreSQL + Redis cache only; warm cacheable reads only; the Redis container's budget is outside the comparison (gap); this 2.2–3.5x range is supported only by the PostgreSQL single-node + Redis cells; do not transfer the range to YugabyteDB, to a cluster, or to open-loop demand

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-15-three-instance-staleness

external read copies / caches · mechanism_supported

Three logical application instances sharing one Redis cache on the write-through + relaxed strategy record about 87% wrong reads, while every single-instance relaxed cell records zero; a private in-memory lease cannot coordinate instances, and a database version token does not remove it. The 87.81% figure belongs to the OWNED model (o:opt:red:wth:rel); the legacy model's cell is 87.24%.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-04-study05-db-only-cache

Limits: single trial, one machine, small scale; generality unproven; the 87%-vs-0% separation is far outside the ~20% noise floor, but the exact rate is one trial; the signed all-green analysis swapped the two models' rows; this claim uses the report and JSON attribution

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v2-16-strict-freshness-read-cost

external read copies / caches · single_run_directional

The strict-freshness contract cost nothing measurable in READ throughput: every controlled strict/relaxed pair differs by between -14.2% and +12.3%, inside the run's ~20% single-trial noise floor. This claim measures read throughput only: it does not quantify the write-path cost of a strict protocol, and it must never be restated as 'strict freshness is free'.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Confounds: conf-06-claim12-full-noise-range, conf-04-study05-db-only-cache

Limits: read throughput only; one trial per measurement; the range is noise, not an error bar; small scale, closed loop

Source: studies/05-cache-consistency/reports/20260921T-survey3.md · run 20260921T-survey3 · tag run/05-cache-consistency/20260921T-survey3

v3-01-churn-crosses-real-ttl

external read copies / caches · single_run_directional

A sustained 1801 s churn phase against the fixed 300 s TTL crossed 6.0 real TTL boundaries with 0 wrong reads in 2,879,077 blended reads and 380,824 writes; the hard-expiry counter is 0 and only probabilistic early expiry fired (37,063), so an entry was never observed to be removed by its hard TTL. This is scoped mechanism and duration evidence, not a general cache result.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one PostgreSQL + Redis cell at scale small; one sustained measurement, no repeat; hard expiry fired zero times; that clause is an open gap, not a confirmation; closed loop, so read volumes are a floor; the 5x-for-120 s burst has no write-rate knob and was not run

Source: studies/05-cache-consistency/reports/20260923T1100Z-churn1800-through.md · run 20260923T1100Z-churn1800-through · tag run/05-cache-consistency/20260923T1100Z-churn1800-through

v3-02-churn-600-through-vs-aside

external read copies / caches · single_run_directional

At 601 s the write-through and write-aside churn cells both crossed 2.0 real TTL boundaries with 0 wrong reads (1,467,721 reads / 197,196 writes for write-through; 1,364,997 reads / 182,282 writes for write-aside), and neither strategy is claimed to win: the study has no head-to-head fill/publication ratio.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: two separate single runs, not a controlled head-to-head pair; write-aside numbers come from a sibling run with its own digest; both are single trials; db-only framing; the cache container is outside the resource comparison here; hard expiry fired zero times in both cells

Source: studies/05-cache-consistency/reports/20260923T1020Z-churn600-through.md · run 20260923T1020Z-churn600-through · tag run/05-cache-consistency/20260923T1020Z-churn600-through

v3-03-equal-total-framing-labelled

external read copies / caches · single_run_directional

Cache resource accounting now has a separately labelled equal-total arm: with the same design and topology the warm cacheable read is 11k ops/s under equal-total framing and 8.0k under db-only, the framing is recorded in every result, and the two arms are never pooled.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run per framing, one host, scale small; the two arms are single trials; the value of the arm is the label and separation, not a throughput ratio; the difference is inside the run's spread and is not attributed

Source: studies/05-cache-consistency/reports/20260923T1150Z-res2-equaltotal-pg.md · run 20260923T1150Z-res2-equaltotal-pg · tag run/05-cache-consistency/20260923T1150Z-res2-equaltotal-pg

v3-04-yb-single-strict-relaxed

external read copies / caches · single_run_directional

On YugabyteDB 1-node (RF=1) one run recorded the relaxed write-through app-99 read at 596 ops/s against the strict cell's 514; the spread is large, this is a single run, and no freshness winner is attributed.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run per freshness policy on one engine; the strict/relaxed direction is not repeated; single endpoint (RF=1); this is not a cluster result; app-99 phase is a small mixed sample (spread 29.5% relaxed, 15.9% strict)

Source: studies/05-cache-consistency/reports/20260923T1152Z-res2-yb1.md · run 20260923T1152Z-res2-yb1 · tag run/05-cache-consistency/20260923T1152Z-res2-yb1

v3-05-yb-cluster3-strict-relaxed

external read copies / caches · single_run_directional

On YugabyteDB 3-node (RF=3), where the client spreads operations over all three endpoints, one run recorded the strict write-through app-99 read at 942 ops/s against the relaxed cell's 846 - the opposite direction from the 1-node pair - with each cell a single run and no engine ranked.

Trials: 1 whole-run replication · 1 fresh-load trial per cell · 1 inner iteration

Limits: one run per freshness policy; direction is not repeated and differs from the 1-node pair; three containers on one host, no real network; endpoint spread is round-robin by construction; app-99 phases have large spread (50.4% relaxed, 49.4% strict)

Source: studies/05-cache-consistency/reports/20260923T1157Z-res2-yb3.md · run 20260923T1157Z-res2-yb3 · tag run/05-cache-consistency/20260923T1157Z-res2-yb3

v3-gap-01-study05-remaining-regimes

external read copies / caches · gap · partially superseded by v4-gap-01-physical-placement-unverified, v4-gap-02-endpoint-distribution-partial · coverage gap + instrument gap

Moved to successor claims: v4-gap-01-physical-placement-unverified, v4-gap-02-endpoint-distribution-partial.

Still open in this claim: hard-TTL expiry never observed removing an entry; medium scale beyond the harness maximum of 3,000 people; open-loop arrival demand.

Statement below is unchanged from its source registry; the moved dimensions are not restated as open here.

Study 05's remaining unmeasured regimes are explicit: the hard TTL never removed an entry in a measured cell (hard expiries 0 in all three churn cells), medium scale beyond the harness maximum of 3,000 people was not run, the tablet/leader placement distribution is absent, and open-loop arrival demand was not run.

Trials: 0 whole-run replications · 0 fresh-load trials per cell · 0 inner iterations

Limits: these are coverage gaps, not null results; medium scale is an order of magnitude beyond the current generator; placement verification needs an instrument the runner does not have; open-loop demand is owned by task-20260922T200831Z-study05-v2-open-loop

Source: studies/05-cache-consistency/reports/20260923T1100Z-churn1800-through.md · run 20260923T1100Z-churn1800-through · tag run/05-cache-consistency/20260923T1100Z-churn1800-through

19.7 Retired predecessor claims

- `claim-gap-04-study02-03-validation-open` — Retired in v1->v2; the validation it described completed 2026-09-22. Kept in the v3 chain so the retirement history stays readable.
- `v2-gap-04-study05-unmeasured-regimes` — Superseded by v3-01..v3-06 plus v3-gap-01: real-TTL churn, repeated trials, equal-total accounting and YugabyteDB/cluster cells are now measured; the still-open regimes are retained explicitly in v3-gap-01.
- `v2-gap-05-colocation-unverified` — Retired: its clause 'Study 05's placement pair was never executed' is contradicted by v3-06, which executed the key-locality pair. The physical-placement clause is carried forward, narrowed, in v4-gap-01. *Closed:* Study 05 key-locality pair executed (v3-06).
- `v2-gap-07-real-network-balanced-endpoints` — Retired: its clause 'balanced client access across cluster endpoints is not proven' is contradicted for the 2026-09-23 Study 05 cells by v3-05, whose client spreads over all three endpoints. The remaining endpoint-distribution and no-real-network clauses are carried forward, narrowed, in v4-gap-02. *Closed:* Study 05 three-node endpoint spread (round-robin over three endpoints).